

テーマ: プライベートで安全な連合学習

(英題: A Study on Private and Secure Federated Learning)



社会情報ネットワーク講座 合意情報学分野

指導教員: 伊藤孝行先生

加藤 郁之

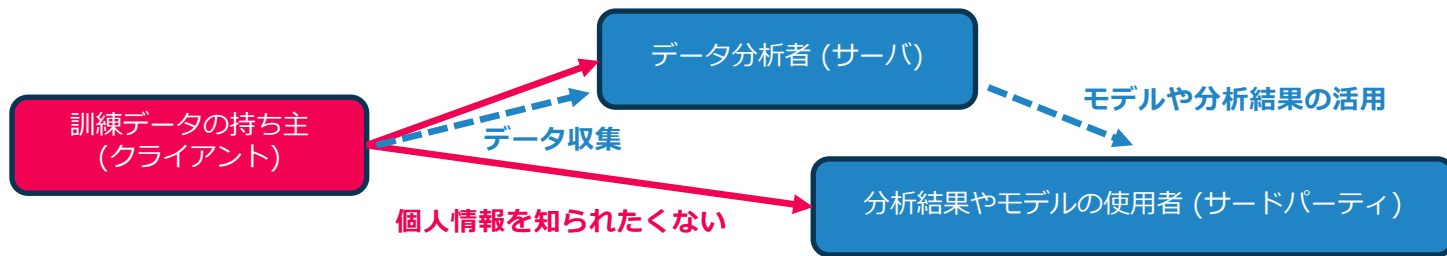
構成

- インTRODクシヨN (Section 1-2)
 - FLについて, FLにおけるPrivacyとSecurityの課題
 - 重要なプライバシー保護技術
 - 差分プライバシー (DP) ([研究成果A, 付録])
 - Trusted Execution Environment (TEE) ([研究成果B, 付録])
 - 本博士論文の貢献
- 研究成果 (Section 3-5)
 - [研究成果1] ULDP-FL: ユーザレベルの差分プライバシー保護型連合学習
 - [研究成果2] OLIVE: サーバサイドTEEを用いた脆弱性に対しても安全な連合学習
 - [研究成果3] Verifiable LDP: 連合型分析における悪意のあるクライアントに対する防御手法
 - (発表スライドショーでは技術的詳細のスライドの一部をスキップしています)
- まとめ/発表文献 (Section 6-7)

個人データとプライバシー

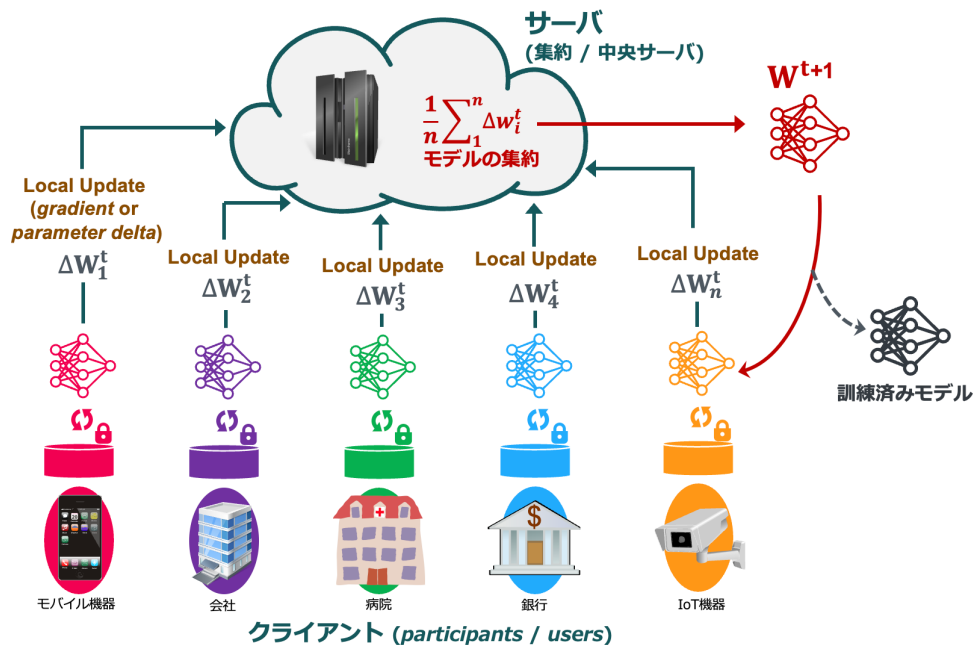
*個人データとは、識別可能な個人に関連するあらゆる情報を指す (欧州委員会より)

- 機械学習を中心に**個人データ***の活用が大規模に行われている
 - Personal data is the new oil of the Internet and the new currency of the digital world (2009, Meglena Kuneva, EU Commissioner for Consumer Affairs)*
- 一方で、データの**プライバシー**への懸念が高まっている
 - 企業が個人データを大量に収集して活用している
 - キーボードの入力履歴から補完ための予測器を作る (Appleなど)
 - 購買履歴から商品の推薦をするための予測器を作る (Amazonなど)
 - 法律による保護も行われ始めている
 - GDPR (EU一般データ保護規則), CCPA (カリフォルニア州)
 - 本人が自身の個人データの削除を個人データの管理者に要求できる, 等



連合学習 (Federated Learning, FL)

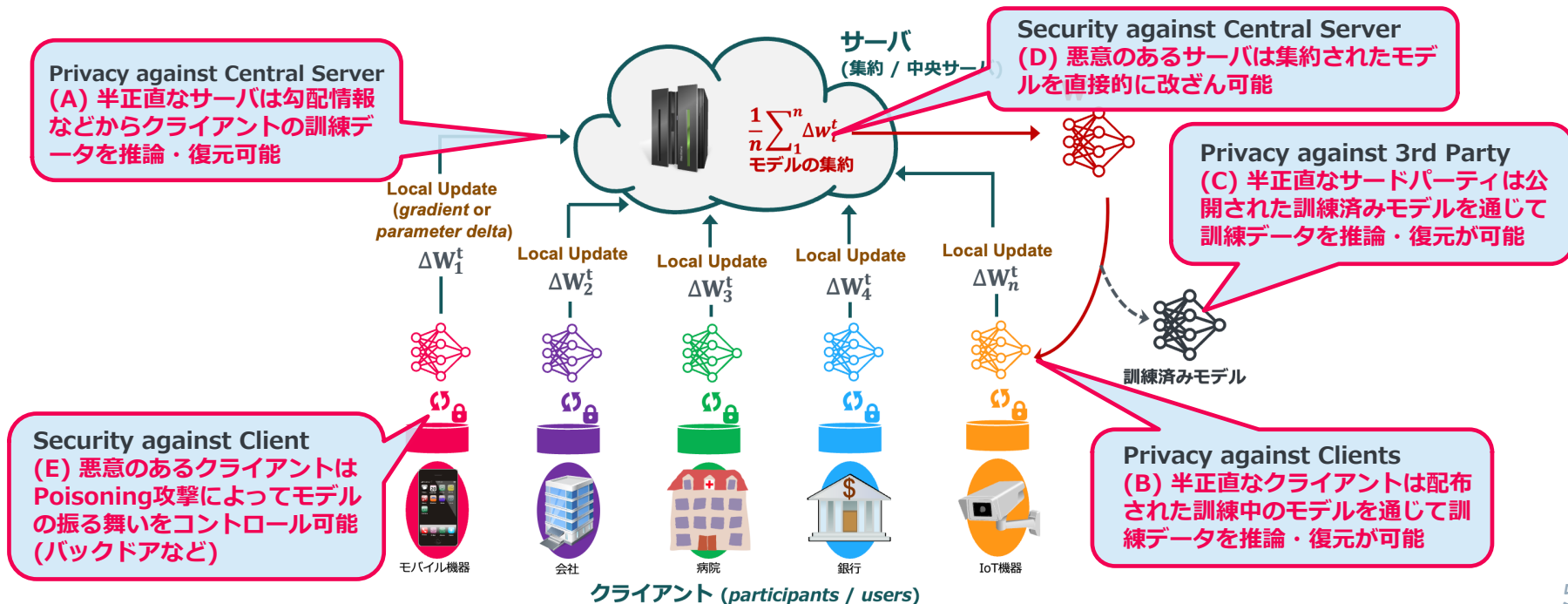
- FLは2016年にGoogleが発表したプライバシー保護型機械学習スキーム
- 訓練データをもつ複数の**クライアント**と1つの**集約サーバ**が協調して1つのモデルを学習する
- 典型的なアルゴリズム -- **FedAVG**
 - クライアントデバイス上でモデルの最適化を行うDistributed SGDがベース
 - 勾配の代わりに、ローカルで反復的に最適化したモデルのパラメータの差分を交換するのがFedAVGの特徴
 - 中央サーバは、訓練中のモデルの配布、モデルの集約を繰り返し行う
 - モデルが収束するまで、配布、学習、集約を繰り返す
- 最も嬉しい点: サーバは生の訓練データを収集する必要がない
- 単純なタスク(頻度推定など)を含む**Federated Analytics(FA)**も登場



Cross-device/Cross-silo FL

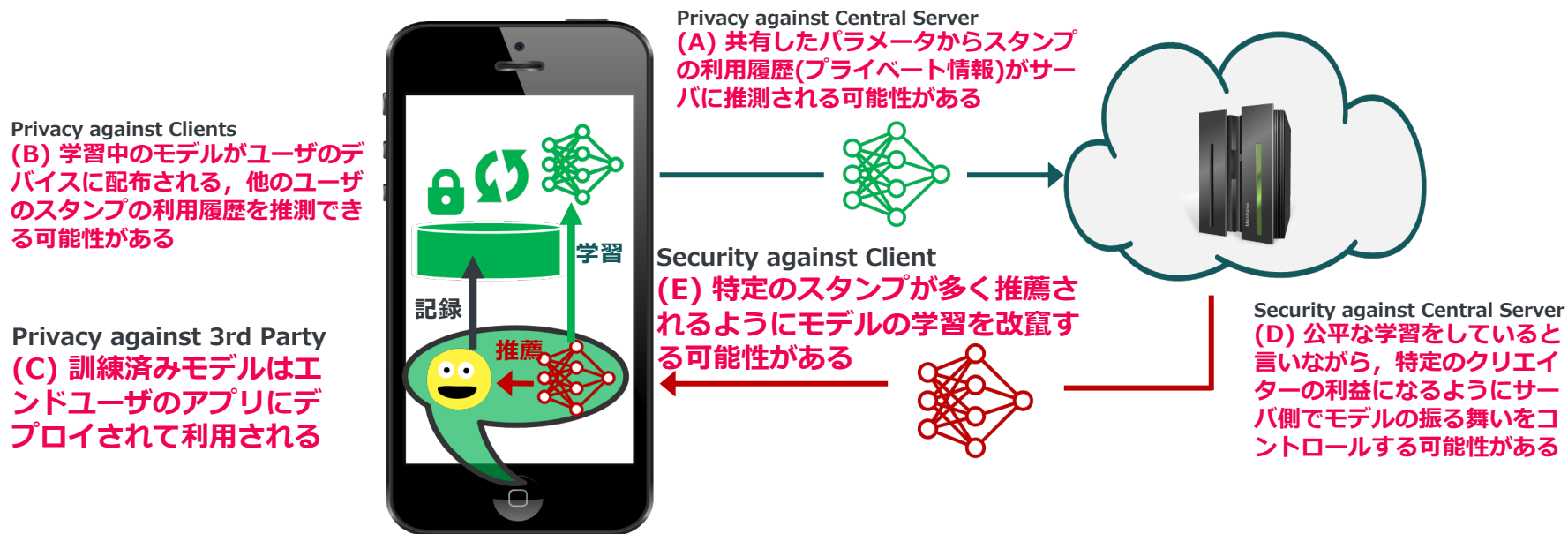
プライバシーとセキュリティの課題

- 実はFLは単体では十分な**プライバシー**を提供しない **(A,B,C)** , さらに, 分散的な設計上, 通常のMLよりも**セキュリティ**リスクを伴う **(D,E)**



プライバシーとセキュリティの課題

- シナリオ例: **LINEアプリ**のFederated Learningを使ったスタンプ推薦 [WP1]
 - ※ 実際にはLocal DPを組み合わせている

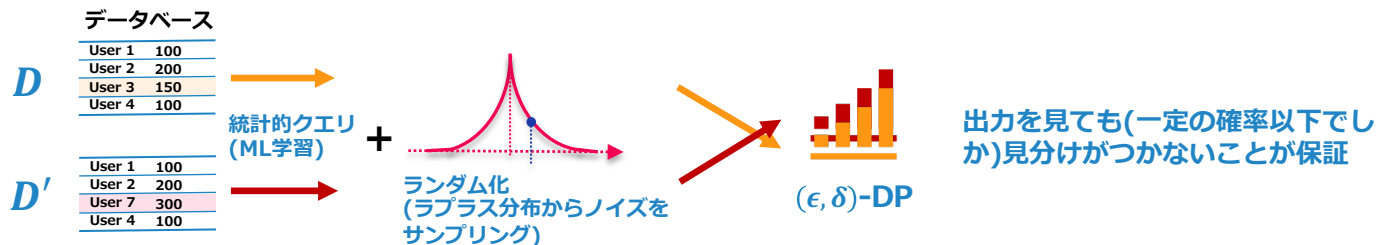


本論文の目的

- FLに欠けている以下の5つのプライバシー/セキュリティ要件をそれぞれ防御する方法を明らかにしたい
 - A. Privacy against Central Server
 - B. Privacy against Clients
 - C. Privacy against 3rd Party
 - D. Security against Central Server
 - E. Security against Client
- 特に、近年登場し注目を集めるプライバシー保護技術である**差分プライバシー (Differential Privacy)**や**TEE**などを、FLにどのように統合することができるか、に焦点を当てる
- それぞれの要件をカバーする3つのフレームワーク[研究成果1,2,3]を提案する

差分プライバシー (Differential Privacy, DP)

- データベース(レコードの集合)に対する数学的なプライバシーの定義 ('06に提案)
- 任意の隣接データベース(レコードが1つ違う)に対する識別不能性をモデル化
 - アルゴリズムA (クエリ+ランダム化)が以下を満たすと (ϵ, δ) -DP
 - $Pr[A(D)] \leq \exp(\epsilon) Pr[A(D')] + \delta$, D と D' は隣接データベース
 - アルゴリズムの出力が D かもしくは D' から得られた確率の差が ϵ でバウンドされる
 - プライバシーバジェット ϵ** が小さいほど D と D' の識別が難しく, よりプライベート
 - クエリ結果を公開するたびに確率の差のバウンドが劣化する(DPの**合成特性**)
- 典型例
 - アルゴリズムAが, Countクエリ + スケールが $1/\epsilon$ のLaplaceノイズが $(\epsilon, 0)$ -DP
 - 高次元のテーブルデータに対する無制限のCountクエリを可能にする [研究成果A, 付録]



差分プライバシー (Differential Privacy, DP)

- 機械学習と差分プライバシー

- DP-SGD:** 勾配の計算をプライベートクエリとみなしてnoisyな勾配を使って最適化→学習されたモデルのパラメータがDPを満たす



User 1	100
User 2	200
User 3	150
User 4	100



学習されたモデルはDPを満たしているため、パラメータも、以降の推論結果も、全てDPが保証されている

- 局所的差分プライバシー (Local Differential Privacy, LDP)

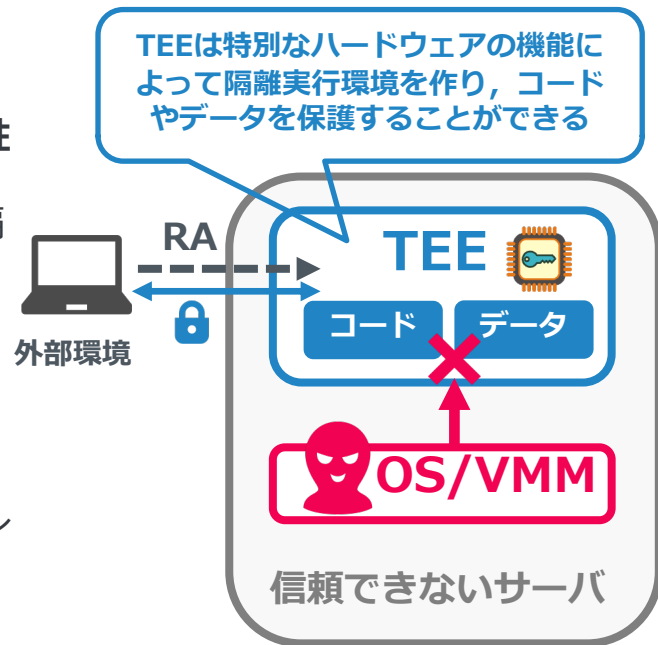
- クライアントのローカル環境でデータをランダム化
- LDPはサーバを信頼しないモデル ⇔ 通常のDP (Central DP)はデータを収集するサーバを信頼



実はすでに結構使われている
AppleのiOS [SIGMOD18]
GoogleのChrome [SIGSAC14]
など

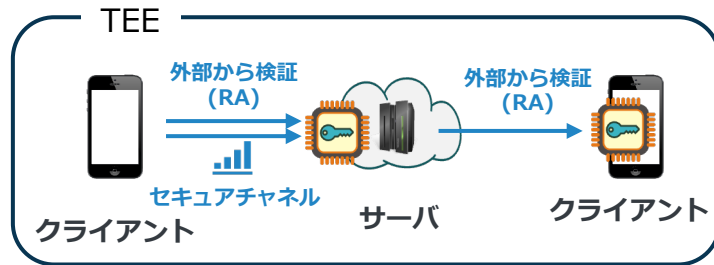
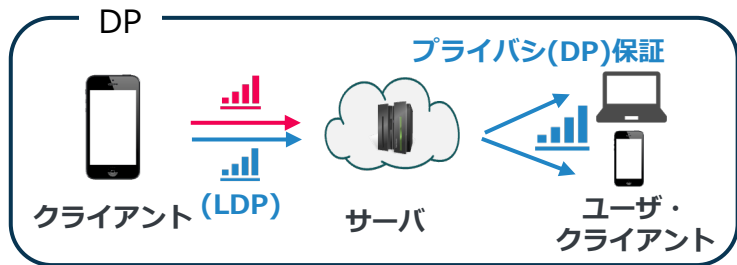
Trusted Execution Environment (TEE)

- **TEE**はリモートマシン上での秘密計算を可能にする
- **Intel SGX** – 最も有名なTEEの実装の1つ
 - 実態はIntel CPUの拡張命令セット
 - 主に、メモリ暗号化やRemote Attestationによって**機密性**、**完全性(コードとデータ)**、**検証性**を提供
 - **Remote Attestation (RA)**は、外部から、実行環境+ 隔離されたメモリの初期状態を検証するプロトコル
 - RA以降はセキュアチャネルを通じて通信可能
- 特徴
 - **ネイティブ並みの実行速度**で**任意の計算**が可能、既存のMPCなどのソフトウェアベースの秘密計算を代替可能
 - TEEを使ったプライベートで効率的なコンタクトトレーシング [研究成果B, 付録]
 - **ハードウェアが必要**、**サイドチャネル攻撃**がたくさんある



DPとTEEが解決する課題

- DPとTEEはFLに欠けているそれぞれのプライバシー/セキュリティ要件を補完する可能性を持つ



現状、クライアント側のTEEはやや強い仮定

	A. Privacy against Central Server	B. Privacy agaist Client	C. Privacy agaist 3rd Party	D. Security against Central Server	E. Security against Client
通常のFL	×	×	×	×	×
DP	CDP × LDP ✓	✓	✓	×	×
TEE (or MPC)	✓	×	×	✓	✓

本博士論文の貢献の概要

- DPとTEEとその代替技術をFLに統合することで, FLのプライバシーとセキュリティを以下のように補完するフレームワークを提案

	A. Privacy against Central Server	B. Privacy against Client	C. Privacy against 3rd Party	D. Security against Central Server	E. Security against Client
通常のFL	✗	✗	✗	✗	✗
DP	CDP ✗ LDP ✓	✓	✓	✗	✗
TEE (or MPC)	✓	✗	✗	✓	✓
ULDP-FL [研究成果1] (FL+DP)	✗ (+MPC ✓)	✓	✓	✗	✗
OLIVE [研究成果2] (FL+TEE(+DP))	✓	✗ (+DP ✓)	✗ (+DP ✓)	✓	✗
VLDP [研究成果3] (FA+MPC)	✓	✓	✓	✗	◐

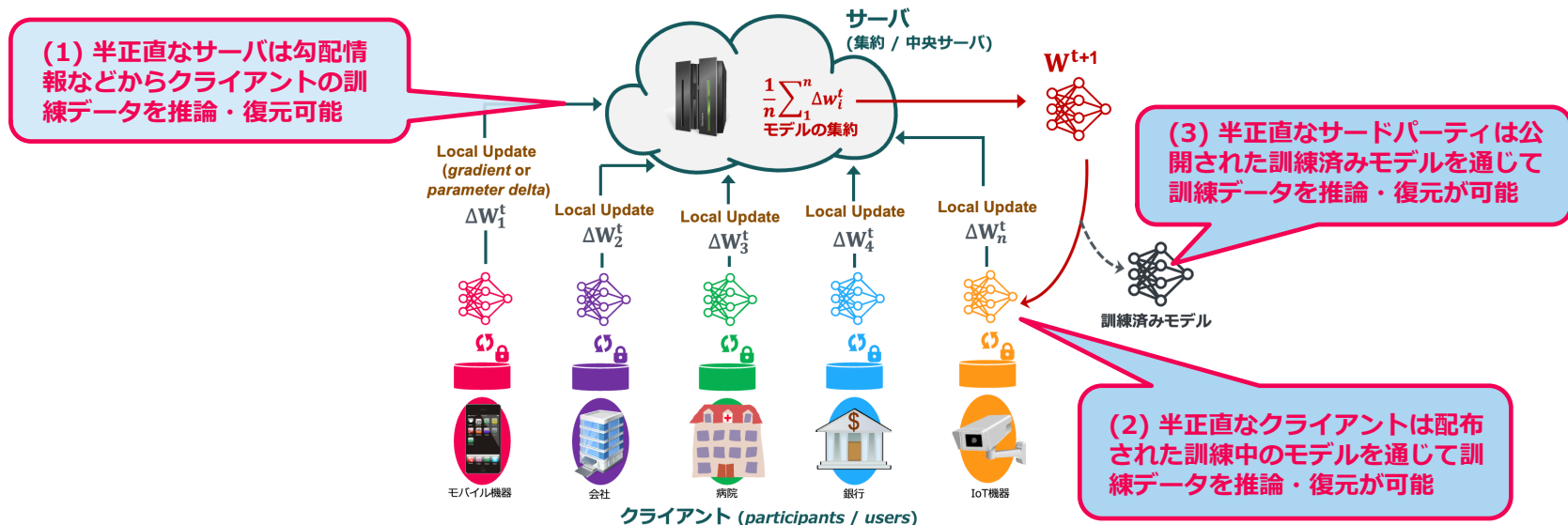
[研究成果1]

ULD_P-FL

ユーザレベルの差分プライバシー保護型ク
ロスサイロFL

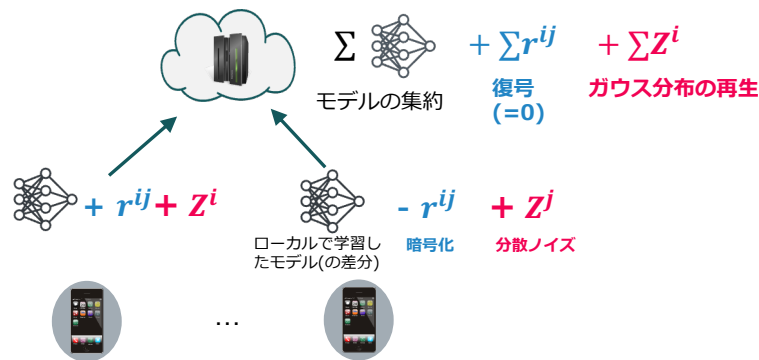
ターゲットとなるプライバシー・セキュリティ要件

	A. Privacy against Central Server	B. Privacy against Client	C. Privacy against 3rd Party	D. Security against Central Server	E. Security against Client
ULDP-FL [研究成果1] (FL+DP)	✗ (+MPC ✓)	✓	✓	✗	✗



差分プライバシー保護型FL

- DPなモデルをFLで学習することで(B,C)を解決可能
- **DP-FedAVG [ICML18]**
 - DPを満たすためにガウシアンノイズを分散環境でシミュレートする
 - Secure Aggregation (足したらキャンセルするペアワイズなマスク(r^{ij})を使って個別の値を暗号化して集約された値のみを公開するMPC手法) を組み合わせることでサーバに対してもDPを保証 [ICML21] ((A)も解決可能)

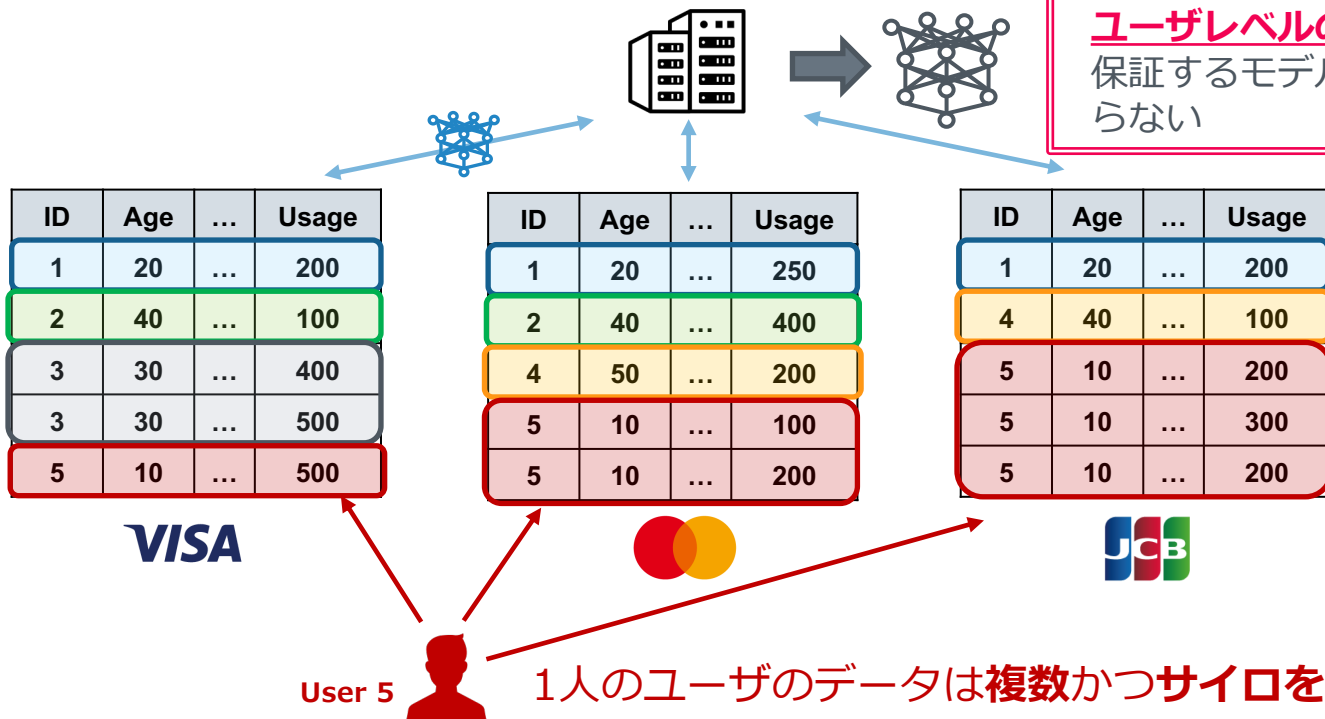


Secure Aggregationによってサーバは個別のクライアントのパラメータを見ることはできず (A), ノイズが付加されDPを満たした集約モデルだけ観測可能

これにより, 訓練済みモデル(C), 訓練中のモデル(B), に対してDPを保証することができる

本研究のゴール

- 一般的なクロスサイロFL
 - 例: 複数のクレジットカード会社



課題

オリジナルのDPとそのアルゴリズム (DP-FedAVGを含む) は1つのレコードレベルのプライバシー保証しかない

ユーザレベルのDPをサイロを跨いで 保証するモデルを学習しなくてはならない

User-level DP

- 通常のDPの定義は、プライバシー保護の単位は各レコード
 - Record-level DPと呼ばれる
 - しかし、しばしばデータベースには**同じユーザの複数のレコードが含まれる**
- User-level DP**
 - ユーザに対してプライバシーを保証するためには、プライバシーの定義をユーザのもつ複数のレコードに拡張することが必要
 - 我々のシナリオではさらに、これを**サイロを跨いで**保証したい

プライバシー保護の単位

隣接データベース D' , D をそれぞれ以下のように定義する必要がある

D' と D は最大1レコード異なる

D' と D は最大1ユーザに属する**全ての**レコードが異なる

Record-level

ID	Age	...	Usage
1	20	...	200
1	40	...	250
1	20	...	200
2	40	...	100
2	40	...	400
3	30	...	400
3	30	...	500

User-level

ID	Age	...	Usage
1	20	...	200
1	40	...	250
1	20	...	200
2	40	...	100
2	40	...	400
3	30	...	400
3	30	...	500

DPの複数レコードへの拡張: Group DP

- Group DP: DPの定義を任意のkレコードに対して拡張
 - 変換定理
 - 任意の ϵ -DP アルゴリズムは 任意のkレコードに対して $k\epsilon$ -DP を満たす
 - 既存のクロスサイロDP-FLの研究[NIPS22][AISTATS23]でもこれが仮定されている
 - 一方で, $\delta > 0$ の時 (デファクトであるガウシアンメカニズムでは $\delta > 0$), この変換は超線形のプライバシー劣化を引き起こす

プライバシーバウンド
が線形に劣化

<http://www.gautamkamath.com/CS860notes/lec5.pdf> [Kamath 2020]

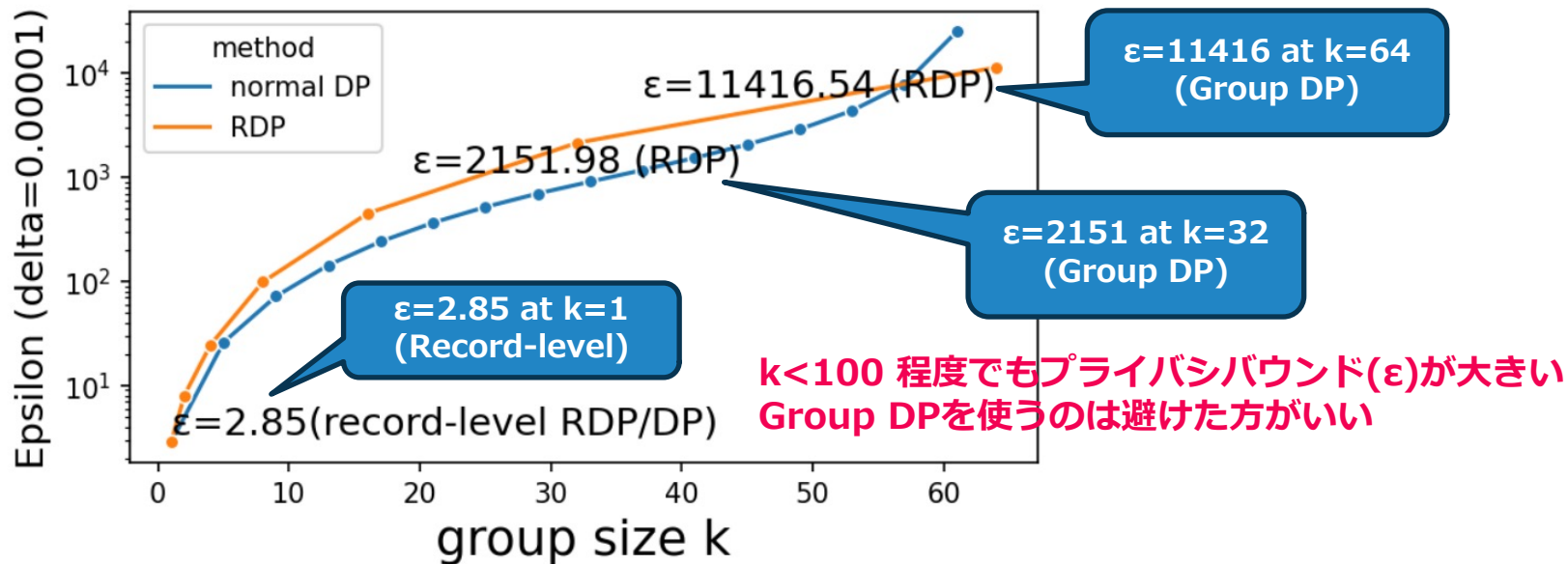
Theorem 10. Let $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ be an (ϵ, δ) -differentially private algorithm. Suppose X and X' are two datasets which differ in exactly k positions. Then for all $T \subseteq \mathcal{Y}$, we have

$$\Pr[M(X) \in T] \leq \exp(k\epsilon) \Pr[M(X') \in T] + \underline{k e^{(k-1)\epsilon} \delta}.$$

プライバシーバウンド
が超線形に劣化

Group DPのプライバシーバウンド

- Record-level DPからk-Group DPへの変換
 - Renyi DP (RDP) (確率分布の差をRenyi Divergence を使って評価するDPの一種, 合成に有利) もGroup DPを計算可能 [CSF17, Prop 2]



プライバシー定義の違い

本質的に異なる

- (Definition 1) Record-level DP
 - 任意の最大1レコードが異なる隣接データベースに対する識別不能性
- (Definition 6) Group DP
 - 任意の最大kレコードが異なる隣接データベースに対する識別不能性
- (Definition 7) Use-level DP
 - 任意の最大1人のユーザの全レコードが異なる隣接データベースに対する識別不能性
 - レコード数に制限がない
- (Proposition 1) k-Group-DPかつ、ユーザごとの最大レコード数がk以下
→ User-level DPを満たす

対象シナリオ

- 脅威モデル
 - 全てのサイロとサーバーは半正直
- プライバシのゴール
 - 訓練済みモデルに対するユーザーレベルの差分プライバシー
- クロスサイロFL
 - 全てのサイロ(2~100程度)は全てのラウンドに参加
 - 全てのレコードは一人のユーザに属し, 事前にサイロを超えて共通のユーザIDが設定されている (Record linkage済み)
 - 全てのユーザのIDの集合はサイロとサーバには既知とする
 - $|S|$: #サイロ, $|U|$: #ユーザ

提案アルゴリズム: ベースライン1

- Group DPに基づくベースライン手法: **ULDP-GROUP- k**
 - [NIPS22]や[AISTATS23]ではこのようなサイロごとのDP-SGDが想定される
- サイロを跨いで各ユーザのレコード数を k 以下に制限し, DP-SGDを使ってモデルを学習してGroup DPに変換
 - グループサイズ k と, どのレコードを使うかの情報を与えられるとする
 - DP-SGDではRecord-levelのDPが得られるので, それを変換定理で k -Group DPに変換する
- DPは以下のように計算可能
 - サイロ内RDP(Record-level, DP-SGDより) $\rightarrow$ RDP's parallel composition [NIPS19](各サイロからサイロ全体へ) $\rightarrow$ RDP's group privacy across-silos [CSF17] $\rightarrow$ ULDP

ここにGroup DPへの変換があるのでバウンドが劣化する

Algorithm 2: ULDP-GROUP- k

Input: S : silo set (silo $s \in S$), η_l : local learning rate, η_g : global learning rate, σ : noise parameter, D_s : training dataset of silo s , C : clipping bound, T : total round, Q : #local epochs, k : group size, γ : sampling rate, $\mathbf{B}$: flags for limit contribution such that for each matrix $\mathbf{b}^s \in \mathbf{B}$ if $b_{u,i}^s = 1$ the user u 's i -th record in silo s is used in training dataset, otherwise the record is excluded from training dataset

```
1: procedure SERVER サーバー側
2:   Initialize model  $x_0$ 
3:   for each round  $t = 0, 1, \dots, T - 1$  do
4:     for each silo  $s \in S$  do
5:        $\Delta_t^s \leftarrow \text{CLIENT}(x_t, C, \sigma, \eta_l, \gamma, \mathbf{b}^s)$ 
6:      $\theta_{t+1} \leftarrow \theta_t + \eta_g \frac{1}{|S|} \sum_{s \in S} \Delta_t^s$ 
7: procedure CLIENT( $x_t, C, \sigma, \eta_l, \gamma, \mathbf{b}^s$ ) クライアント側
8:    $D'_s \leftarrow \text{filter } D_s \text{ by } \mathbf{b}^s$  ユーザのレコードを削除
9:    $x_t^Q \leftarrow \text{DP-SGD}(\theta_t, D'_s, C, \sigma, \eta_l, \gamma, Q)$  通常のDP-SGD
10:  ▷ Algorithm 1 in (Abadi et al. 2016)
11:   $\Delta_{t+1} \leftarrow x_t^Q - x_t$ 
12:  return  $\Delta_{t+1}$ 
```

提案アルゴリズム: ベースライン2

- DP-FedAVGのノイズを調整するだけのベースライン手法: **ULDP-NAIVE**
- アルゴリズムの戦略
 - DP-FedAVGにおけるユーザレベルの敏感度(一人のユーザが出力に与える影響の上限)を計算する $\rightarrow C|S|$
 - DP-FedAVGではレコードごとの敏感度を, パラメータに対するL2クリッピングによって設定
 - その敏感度に応じたノイズを分散的に付加
 - $Gauss(0, I\sigma^2 C^2 |S|^2 / |S|)$
- 直感的には, 一人のユーザが結果全体($\sum_s \Delta_t^s$)に対して100%寄与する可能性があるのでノイズが大きい

Algorithm 1: ULDP-NAIVE

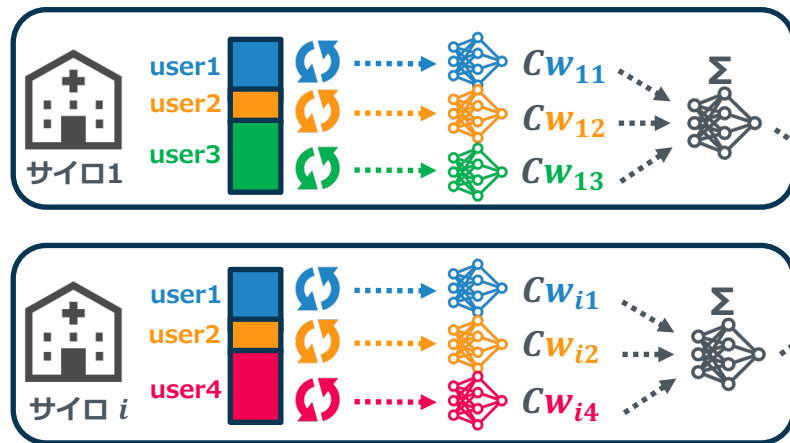
Input: S : silo set (silo $s \in S$), η_l : local learning rate, η_g : global learning rate, σ : noise parameter, C : clipping bound, T : total round, Q : #local epochs

```
1: procedure SERVER サーバー側
2:   Initialize model  $x_0$ 
3:   for each round  $t = 0, 1, \dots, T - 1$  do
4:     for each silo  $s \in S$  do
5:        $\Delta_t^s \leftarrow \text{CLIENT}(x_t, C, \sigma, \eta_l)$ 
6:       /* Summation is performed by secure aggregation */
7:        $x_{t+1} \leftarrow x_t + \eta_g \frac{1}{|S|} \sum_{s \in S} \Delta_t^s$ 
8: procedure CLIENT( $x_t, C, \sigma, \eta_l$ ) クライアント側
9:    $x_s \leftarrow x_t$ 
10:  for epoch  $q = 0, 1, \dots, Q - 1$  do レコードごとに
11:    Compute stochastic gradients  $g_{t,q}^{(s)}$  モデルを学習
12:     $\triangleright \mathbb{E}[g_{t,q}^{(s)}] = \nabla f_s(x_s)$ 
13:     $x_s \leftarrow x_s - \eta_l g_{t,q}^{(s)}$ 
14:     $\Delta_t \leftarrow x_t - x_s$ 
15:     $\tilde{\Delta}_t \leftarrow \Delta_t \cdot \min\left(1, \frac{C}{\|\Delta_t\|_2}\right)$  L2クリッピング
16:     $\Delta'_t \leftarrow \tilde{\Delta}_t + \mathcal{N}(0, I\sigma^2 C^2 |S|)$  ノイズ付加
17:    return  $\Delta'_t$ 
```

DP-FedAVGとの主な違いはここだけ!

提案アルゴリズム: より高度な手法

- もっとユーザレベルの敏感度を小さくする手法: **ULDP-AVG**
 - ユーザレベルの敏感度を制限する. そのために, 各サイロではユーザごとにパラメータを計算してクリッピングする
 - さらに, プライバシ増幅(プライバシ境界をより小さくする)のために, ユーザレベルのサブサンプリングを行う
 - サイロを跨いでユーザごとのパラメータをクリッピングする必要があるので, 合計値が一定になるように以下のように重み付けする



任意のユーザ u に対して
 $\sum_i w_{iu} = 1$ とすることでユ
ーザレベルの敏感度が C

$$\Sigma \text{ (neural network icon) } + \text{Gauss}(0, I\sigma^2 C^2)$$

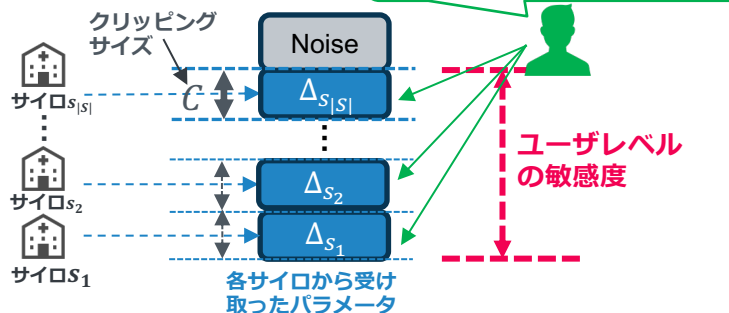
ガウシアンノイズは分散的に載せる

各ラウンドの計算結果が
User-levelの(R)DPを満たす

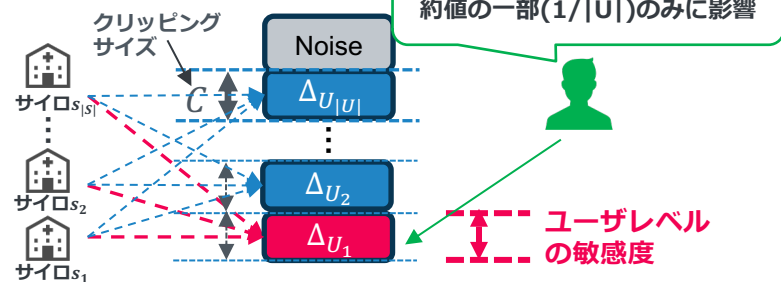
提案アルゴリズム: より高度な手法

- ULDP-NAÏVEと比べて一人のユーザのパラメータの集約値に対する敏感度が大きく違う
 - ULDP-AVGでは $1/|U|$
 - Cross-silo FLでは $|S| \ll |U|$

ULDP-NAÏVE

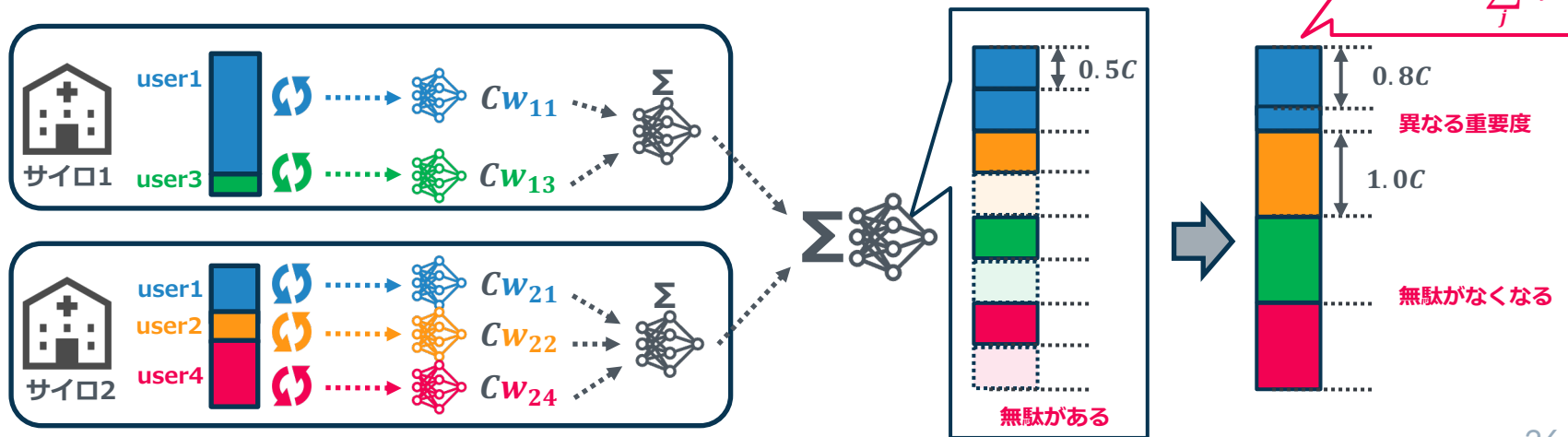


ULDP-AVG



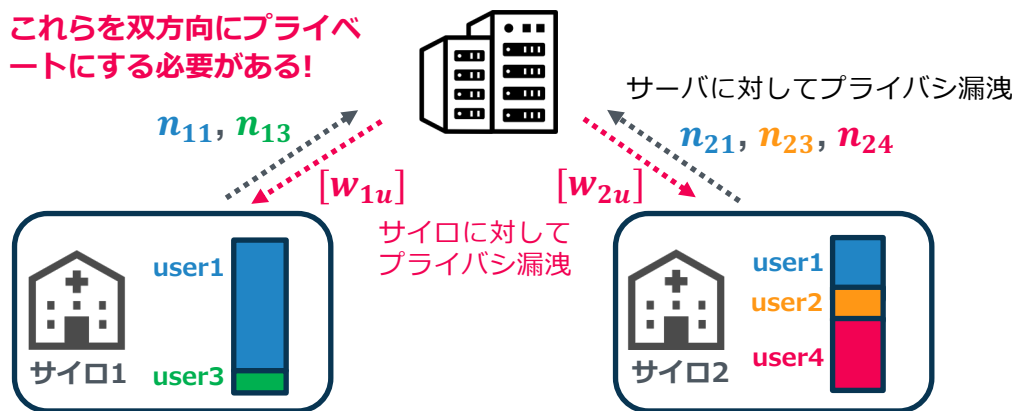
提案アルゴリズム: より高度な手法

- **ULDP-AVG**の精度をさらに上げる: **ULDP-AVG-w**
 - 重み w_{iu} は例えば, 任意のサイロ*i*ユーザuに対して $1/|S_i|$, とできる
 - 下の例では, $w_{iu} = 0.5$
 - 一方で, ユーザのデータに偏りがある場合は, 重みが無駄になる
 - 下の例では, w_{23} , w_{12} , w_{14} などが無駄になっている
 - 収束解析からも重みがバイアス項の大きさに影響することが分かる
 - レコード数に応じて重み付けを決定する $w_{iu}^{opt} = n_{iu} / \sum_j n_{ju}$



重みは双方向にプライバシー漏洩を起こす

- レコード数に応じて重み付けを決定する $w_{iu}^{opt} = n_{iu} / \sum_j n_{ju}$ ためには、重みを決定するプロトコルが必要
- しかし、レコード数(n_{iu})はプライバシー性の高い情報なので集約サーバに知られるのは良くない (最終的なモデルのプライバシーには影響がない)
 - 各サイロは、重み(w_{iu})から他のサイロのデータの分布も推定可能
- 全てのレコード数をサーバに対して、重みをサイロに対して秘匿したい



プライベートな重み付けプロトコル

- サーバもサイロも半正直で共謀しないと仮定
- Private Weighted Sum Aggregation [TCNS21]をベースに以下の3つの技術を組み合わせる
 - Paillier Encryption [TCNS21]
 - Secure Aggregation [CCS17]
 - Multiplicative Blinding (e.g., used in [ACISP07][CDC16])
- ポイント
 - 既存の加法的準同型暗号技術(Paillier[TCNS21])では除算が扱えず、逆数(すなわち、重みを計算するために必要な $1/\sum n_i$)を計算できないが、 Multiplicative Blindingを組み合わせることで、**双方向に秘密値を保護しながら逆数を含む重みを計算**できる
- 提案手法のプライバシ性(サイロが得られる情報とサーバが得られる情報が無意味)と正確性(無視できる確率以外で正しく計算ができる)を示した

[CCS17] Bonawitz et.al, Practical Secure Aggregation for Privacy-Preserving Machine Learning. CCS 2017.

[ACISP07] Damgård et.al, Efficient and Secure Comparison for On-Line Auctions. Australasian Conference on Information Security and Privacy 2007.

[CDC16] Shoukry et.al, Privacy-aware quadratic optimization using partially homomorphic encryption. IEEE Conference on Decision and Control 2016.

[TCNS21] Alexandru et.al, Private Weighted Sum Aggregation. IEEE Transactions on Control of Network Systems 2021.

プロトコルの流れ（各詳細は[付録]に）

1. (サーバと各サイロ) DH鍵交換を通じて、サーバ経由で各サイロペアごとに共有鍵を作成する、サーバはPaillierの公開鍵をサイロに配布する
2. (各サイロ) 共有鍵を共通シードに、共通の乱数列を作成する
3. (各サイロ) 2で得られた乱数列を使ってユーザのレコード数のヒストグラムに対してMultiplicative Blindingを行う
4. (サーバと各サイロ) Secure Aggregationを使ってBlindedヒストグラムを集約
5. (サーバ) Blindedヒストグラムに対して逆数を計算して、(さらに必要であれば、ユーザレベルのサブサンプリングをして重みをゼロ化する)値をPaillier Encryptionによって暗号化する
6. (サイロ) ローカルの学習中にPaillierで暗号化された重みで重み付ける
7. (サーバと各サイロ) Secure Aggregationによってモデルパラメータを集約する
8. (サーバ) 集約されたデータをPaillierの秘密鍵を使って復号してグローバルモデルを更新する
9. (5,6,7,8)を繰り返す

ULDP-AVGの収束解析

- 最初の3つの項は、global learning rateを $|S|$ 倍に選択することで漸近的にサイロ/ユーザを参加者としたFedAVGの収束レートと同じ
- 第4項目はガウシアンノイズの影響で、これはユーザを参加者としたときのDP-FedAVGと同じ
- 最後の第5,6項目はクリッピングによるバイアスから発生するオーバーヘッドを表す
 - オーバーヘッドは、それぞれで計算される勾配の分散が大きい時に大きくなり、i.i.d.の設定だと0になる

Theorem 4 (Convergence of ULDP-AVG). *For ULDP-AVG (Algorithm 4), with assumptions 1, 2 and 3 and $\min_x f(x) \geq f^*$, let local and global learning rates η_l and η_g be chosen as such that $\eta_g \eta_l \leq \frac{1}{3QL\bar{\alpha}_t}$ and $\eta_l < \frac{1}{\sqrt{30QL}}$, we have,*

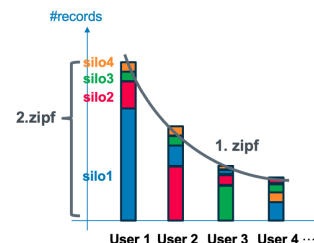
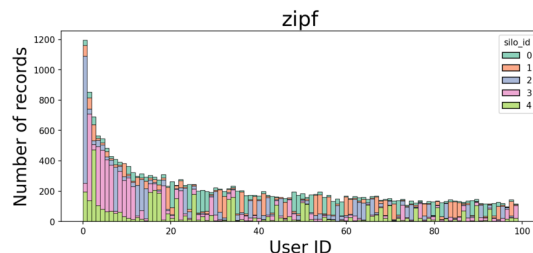
$$\begin{aligned}
 & \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} [\|\nabla f(x_t)\|^2] \\
 & \leq \frac{1}{cT\eta_g\eta_l Q|S|} \left(\mathbb{E} \left[\frac{f(x_0)}{\underline{C}} \right] - \mathbb{E} \left[\frac{f^*}{\bar{C}} \right] \right) \\
 & \quad + \frac{5}{2c} L^2 Q \eta_l^2 (\sigma_l^2 + 6Q\sigma_g^2) + \frac{3\bar{C}L\eta_g\eta_l\sigma_l^2}{2c|S|^2|U|} + \frac{L\eta_g\sigma^2 C^2 d}{2cC\eta_l Q|S||U|^2} \\
 & \quad + A_1 \sum_{t=0}^{T-1} \mathbb{E} \left[\sum_{s \in S} \sum_{u \in U} (|\alpha_t^{s,u} - \tilde{\alpha}_t^{s,u}| + |\tilde{\alpha}_t^{s,u} - \bar{\alpha}_t|) \right] \\
 & \quad + A_2 \sum_{t=0}^{T-1} \mathbb{E} \left[\sum_{s \in S} \sum_{u \in U} (|\alpha_t^{s,u} - \tilde{\alpha}_t^{s,u}|^2 + |\tilde{\alpha}_t^{s,u} - \bar{\alpha}_t|^2) \right] \quad (4)
 \end{aligned}$$

where c is a positive constant, $\bar{C} := \max_{s \in S, u \in U, t \in [T]} \left(\frac{C}{\max(C, \eta_l \|\mathbb{E}[\sum_{q \in [Q]} g_{t,q}^{s,u}]\|)} \right)$, $\underline{C} := \min_{s \in S, u \in U, t \in [T]} \left(\frac{C}{\max(C, \eta_l \|\mathbb{E}[\sum_{q \in [Q]} g_{t,q}^{s,u}]\|)} \right)$, $\alpha_t^{s,u} := \frac{w_{s,u} C}{\max(C, \eta_l \|\sum_{q \in [Q]} g_{t,q}^{s,u}\|)}$, $\tilde{\alpha}_t^{s,u} := \frac{w_{s,u} C}{\max(C, \eta_l \|\mathbb{E}[\sum_{q \in [Q]} g_{t,q}^{s,u}]\|)}$, $\bar{\alpha}_t := \frac{1}{|S||U|} \sum_{s \in S} \sum_{u \in U} \tilde{\alpha}_t^{s,u}$, $A_1 := \frac{G^2}{c\underline{C}|U|T}$ and $A_2 := \frac{3L\eta_g\eta_l Q G^2}{2c\underline{C}|U|T}$. (Proof is available in Appendix.)

実験設定

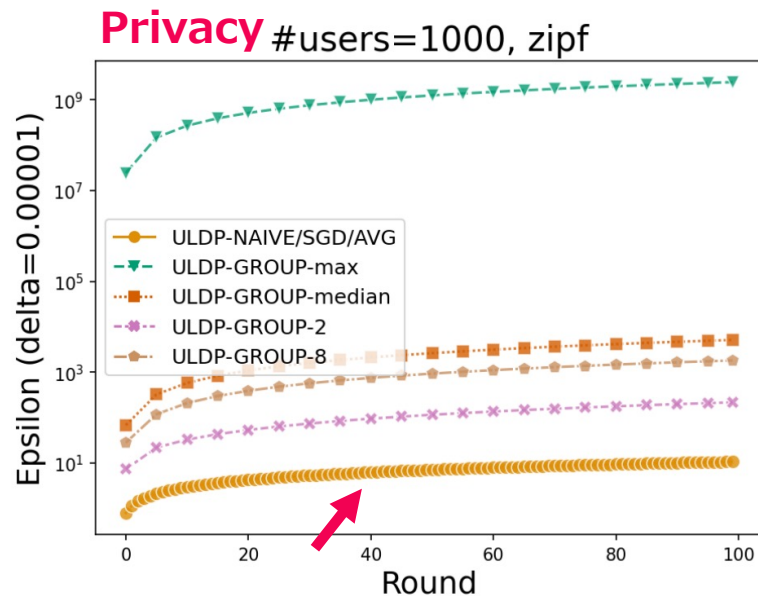
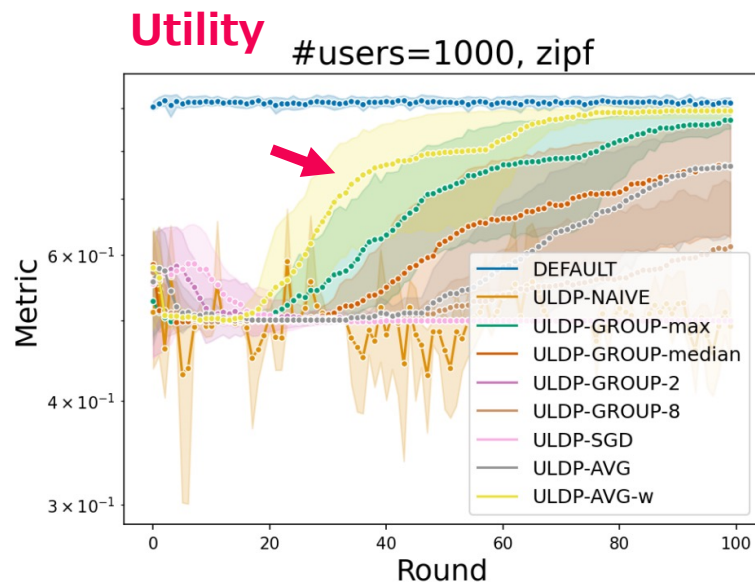
- 比較手法
 - ULDP-NAIVE, ULDP-GROUP(グループサイズは{2, 8, レコード数の中央値, 最大値}), ULDP-SGD, ULDP-AVG, ULDP-AVG-w(重みを最適化したもの), DEFAULT (プライベートでないFedAVG)
- 固定の noise-multiplier $\sigma = 5.0$, $\delta = 10^{-5}$
- 他のハイパーパラメータ η_l, η_g, C, Q
- データセット (Zipf分布などを使って独自にユーザを割り振っている, 詳細は論文の3.5.1)
 - Kaggle上のクレジットカードの不正検出 (Creditcard)
 - Flamby[NIPS21]より2種の医療データセット(HeartDiseaseとTcgaBrca)
 - MNIST

ユーザの割り振り例
#Records 24000
#Users 100
#Silos 5



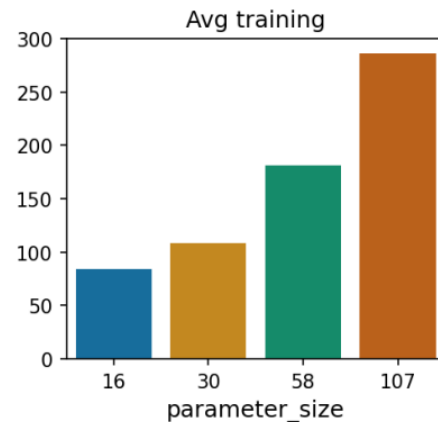
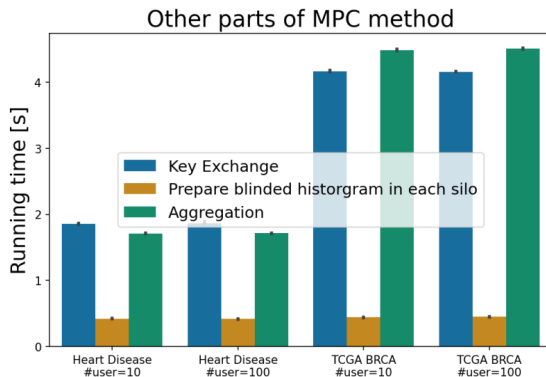
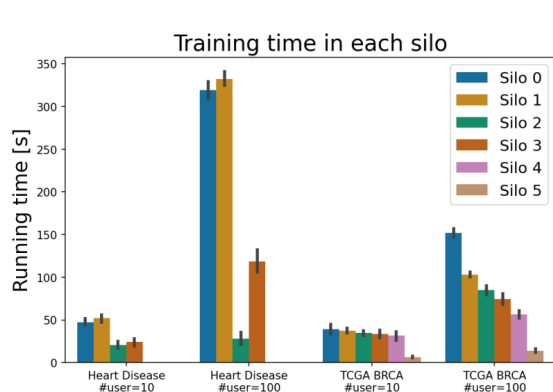
実験結果: (User-levelの) Privacy-utility trade-offの大きな改善

- Creditcard, $|S|=5$, $|U|=1000$ ユーザの平均のレコード数=25
 - 比較手法と比べて, **ULDP-AVG-w**の収束が早く, 精度も高い (左図, 黄色)
 - Group DPベースの手法でかつ近い精度のものは結局プライバシーが悪い (右図)



実験結果: プライベートプロトコルは小さな学習規模では動作する

- HeartDiseaseとTcgaBrcaはモデルの次元が100以下と小さい
 - 普通のPython(3.92), シングルスレッドで実装
 - このくらいのスケールだと現実的な時間(1ラウンド300秒くらい)で終わる(左図)
 - ボトルネックはローカルの学習の計算, Paillier暗号の計算(有限体上の巨大数のスカラ倍の計算に時間がかかる)(中央図)
 - パラメータサイズに対して線形に時間が伸びる, **スケーラビリティが課題**(右図)



※ 有用性の劣化はない:集約プロセスは, Paillier暗号の公開鍵に付随するモジュラスの有限体上で計算する必要があるけど, 固定小数点への近似精度を1e-10にしたら学習結果は全く同じ。(Pythonのfloat64の精度は, 1e-15程度, float32は1e-7程度)

まとめと今後の課題

- ユーザがサイロを跨いで存在する一般的なクロスサイロFLにおいて、より現実的な設定でプライバシー/セキュリティ要件(**A,B,C**)を満たすため、サイロを跨いだUser-level DPを保証するための効果的なアルゴリズムの設計と評価を行いました
- さらに、有用性を高めるためのプライベートな重み付けプロトコルを設計して効率性を評価しました
- 今後の課題
 - 提案フレームワークをより一般的なプライバシー定義へと拡張する、例えば、異なるユーザが異なるプライバシー要求を持つことを可能にする personalized user-level DP
 - 提案MPCプロトコルはディープなモデルに対してはまだ全然遅い

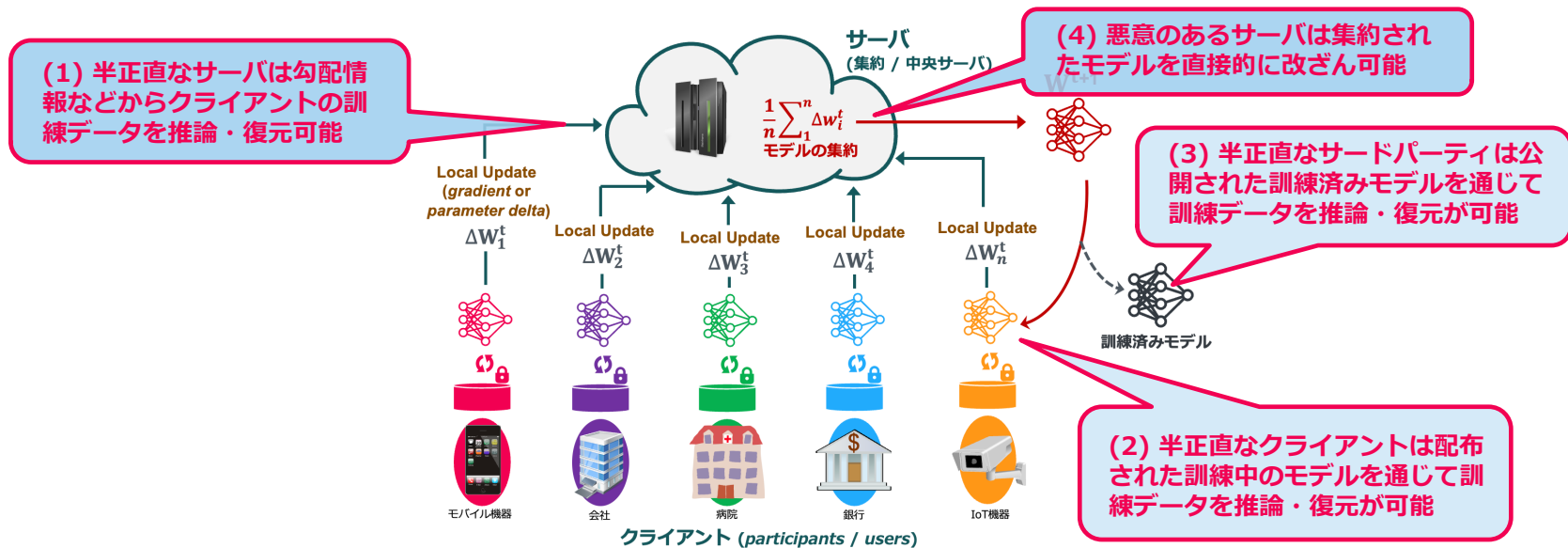
[研究成果2]

OLIVE

サーバサイドTEEを用いた脆弱性に対し
ても安全な連合学習

ターゲットとなるプライバシー・セキュリティ要件

	A. Privacy against Central Server	B. Privacy against Client	C. Privacy against 3rd Party	D. Security against Central Server	E. Security against Client
OLIVE [研究成果2] (FL+TEE(+DP))	✓	✗ (+DP ✓)	✗ (+DP ✓)	✓	✗



サマリー

サーバーサイドTEEを使ったFL

クライアントたち



信頼できないサーバ

Side-Channels



TEE



秘密で安全な集約操作

w^{t+1}

- FLにおいて**サーバサイドTEE**を使うという方向性は、FLのプライバシー/セキュリティに関するメリットが大きいため、とても注目されている。
- しかし、サーバサイドTEE (Intel SGX)にはいくつかの有名な**サイドチャネルが存在し**、それを利用された場合に本当にプライベートなのかは不明
- 本研究では、実際に**プライバシー攻撃が可能であることを実証**して防御機構の必要性をハイライトする
- それに対する効率の良い**防御機構を提案**する

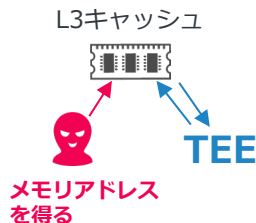
サーバサイドTEEを使うことで、TEEに対してセキュアにパラメータを共有することができ、秘密(A)かつ安全(D)に集約操作が可能

さらに、DPを組み合わせることで(B,C)も満たせる

最近多くの研究がこれを仮定

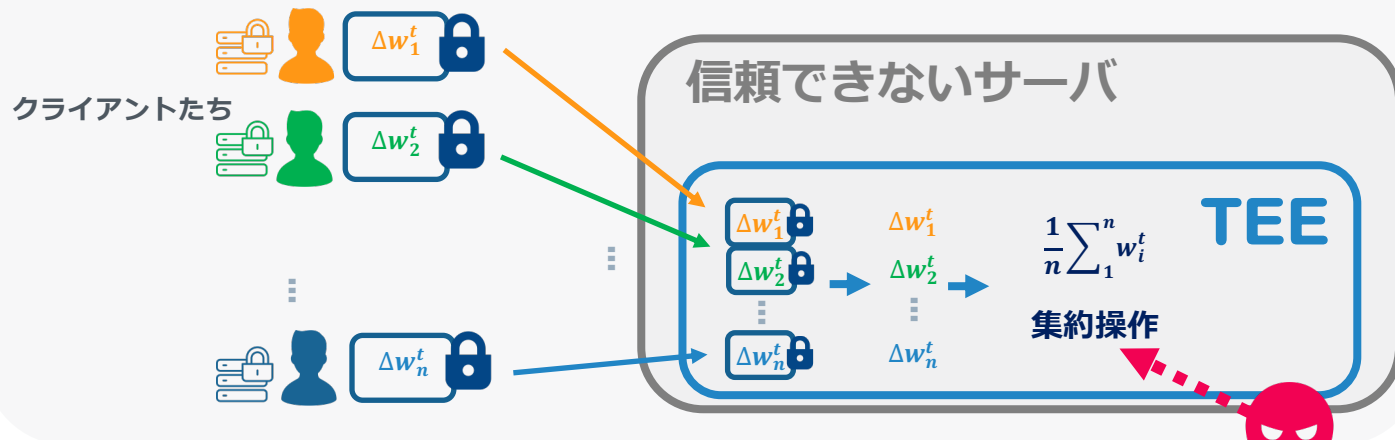
TEEのサイドチャネル

- サイドチャネルからメモリアクセスパターンが漏洩する
 - **メモリ暗号化はメモリの中のデータを保護するが、メモリアクセスパターンは保護しない**
 - メモリアクセスパターンから機密情報が漏洩するかはアプリケーション依存
- サイドチャネルの例
 - 共有キャッシュを使ったサイドチャネル (Prime+Probe)
 - SGXからAESの共有鍵を取り出すことに成功 [CH17]
 - ページフォールトを使ったサイドチャネル (制御パス推定)
 - 文字列, JPEG画像の復元 [SP15]
 - 物理アドレスバスをプローブを使ってモニタリング
 - その他の未知の攻撃によって漏洩する可能性がある(新しい攻撃がどんどん見つかる, かつCPUベンダのセキュリティの対象外とされることが多い)



サーバサイドTEEを使ったFLの課題

サーバサイドTEEを使ったFL



サイドチャネルを通じて攻撃者がメモリアクセスパターンを観測できる

課題: FLにおけるメモリアクセスパターン漏洩の影響は不明

- 具体的にどのような攻撃が可能で、プライバシー侵害は起こるのか
- この攻撃リスクに対する現実的な防御策は何か？

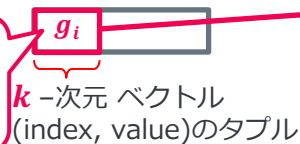
FLの集約操作から発生するデータ依存のメモリアクセスパターン

- パラメータが密であるならば、集約操作のメモリアクセスはデータ非依存である

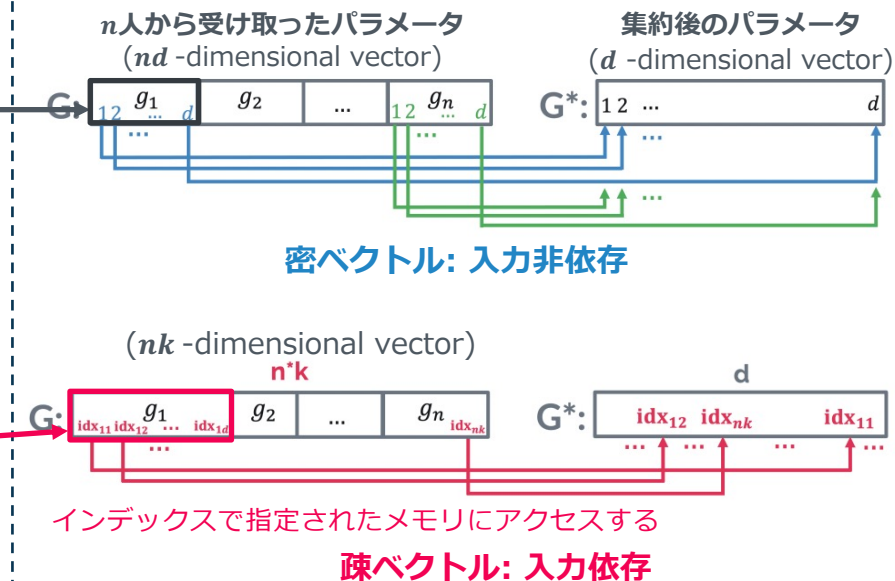


- パラメータが疎であるとき、メモリアクセスパターンは選択されたパラメータのインデックスを漏らす

(データ依存な) Top- k スパース化は、より良い通信コストと有用性のために、FLで頻繁に使用される [NIPS21a]

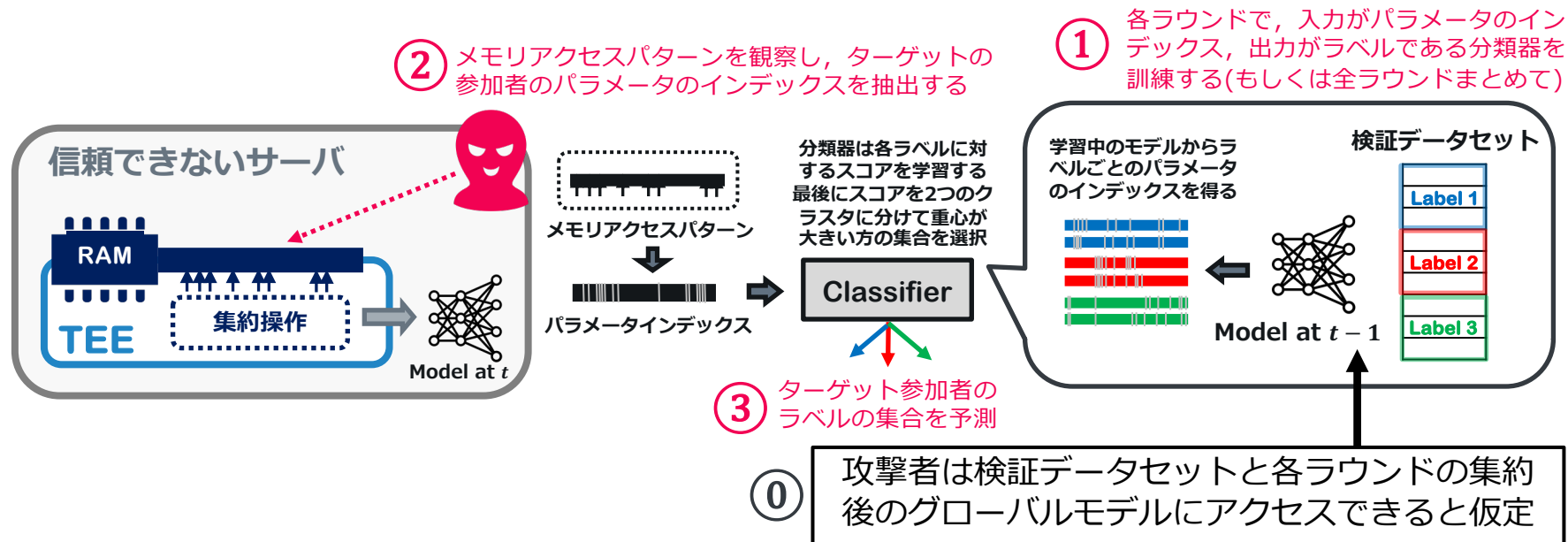


TEE内でのサーバサイドの集約操作におけるメモリアクセス



攻撃の概要

- メモリアクセスパターンから漏れる情報がプライバシーリスクにつながることを示すために攻撃を設計する
- 攻撃者はターゲット参加者の持つプライベートなラベル集合を推測する



実験設定

- データセット
 - MNIST と CIFAR100 と Purchase100 (テーブルデータ)
- スパース化: Top-10% スパース化 (絶対値の大きいパラメータを残してスパース化)
- 評価指標 (攻撃者は全てのFL参加者に対して推論攻撃を行い, その精度を計測する)
 - **All**: 攻撃者によって推定されたラベル集合とユーザのラベル集合が完全に一致した割合
 - **Top-1**: 一番高いスコアを得たラベルがユーザのラベル集合に含まれる割合
 - これが最も弱いプライバシー漏洩
- 各参加者の最大のラベル数(訓練データの持つラベルの集合)をコントロール(難易度)
 - 下の図のように, 参加者1の持つデータセットのラベルは1か3で, 参加者2は3か5か8, 最大ラベル数は3



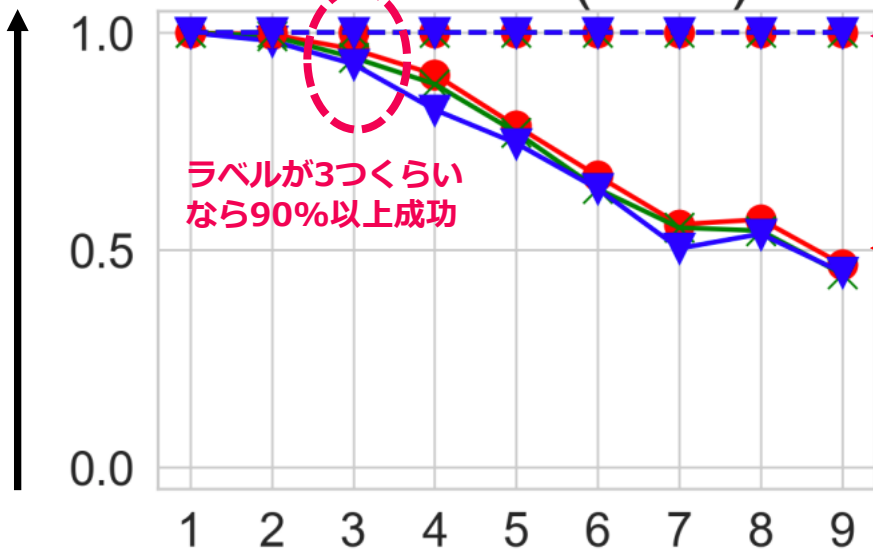
実験結果：攻撃は成功する



NN: 分類器がニューラルネットワーク
NN-single: 全ラウンドをバッチ的に学習する
Jac all: 分類器がJaccard係数

推論攻撃の成功率

CIFAR10 (MLP)



ラベルが3つくらい
なら90%以上成功

Top-1 ラベルを推測するのは非常に簡単



{3,5,8}

5
Top-1の予測ラベル
が含まれる



ラベルの集合が完全に推測することも高い精度で可能



{3,5,8}

{3,5,8}

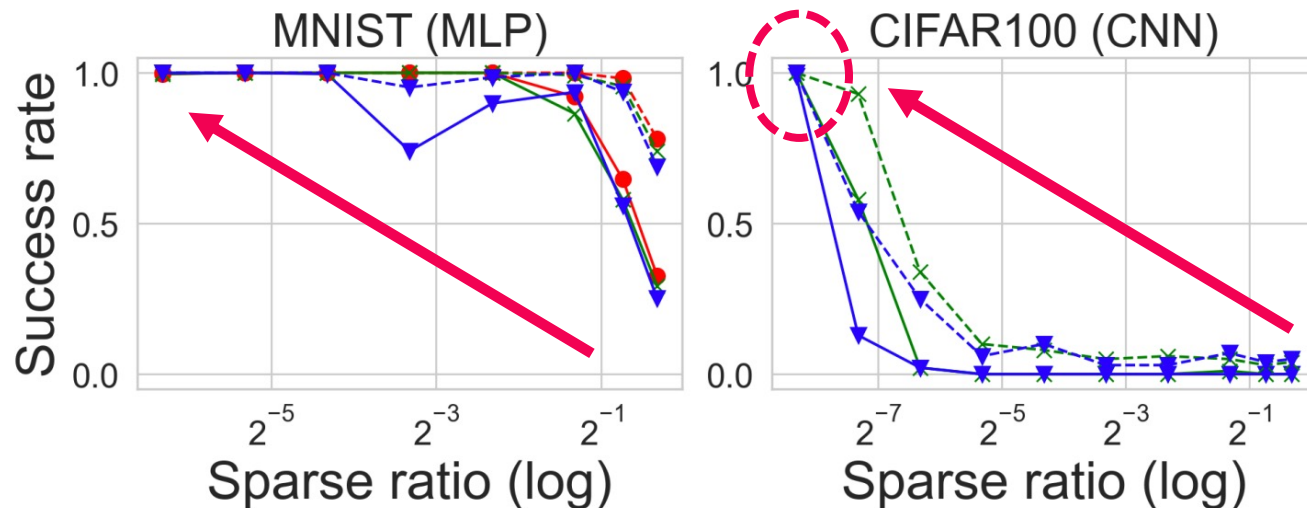
予測集合が完全にマッチ



各参加者の最大のラベル数
(攻撃の難易度)

スパースであるほど攻撃は簡単

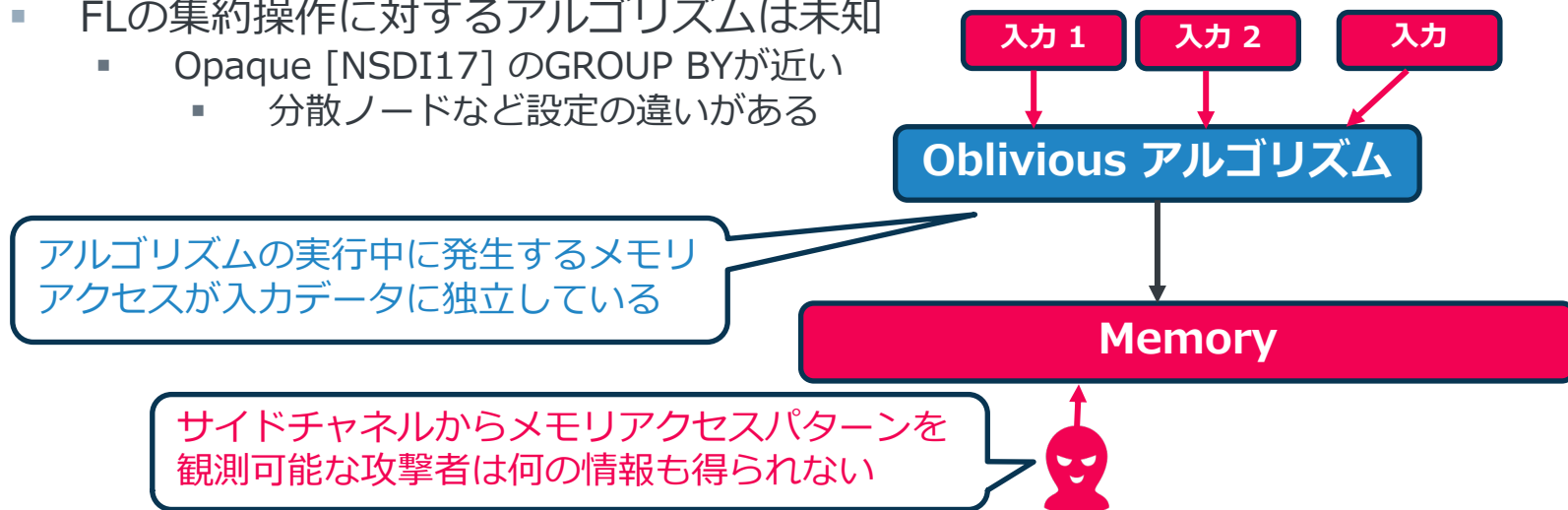
参加者が持っているラベル数は2個で固定



100個のクラスラベルがあったとしても(CIFAR100), スパース率が0.3%程度の時,
攻撃はほぼ100%成功する
(FLで使われる一般的なスパース率は, 例えば, 0.1% [NIPS21b])

Obliviousアルゴリズム

- この攻撃を防御するためには、メモリアクセスパターンを入力データに非依存にしていれば十分
- Oblivious アルゴリズム** は、メモリアクセスパターンの分布が入力に対して独立しているアルゴリズム (例: Obliviousソート)
- FLの集約操作に対するアルゴリズムは未知
 - Opaque [NSDI17] のGROUP BYに近い
 - 分散ノードなど設定の違いがある

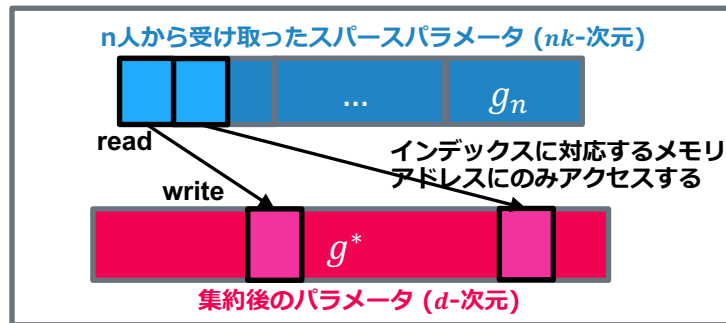


ベースラインObliviousアルゴリズム

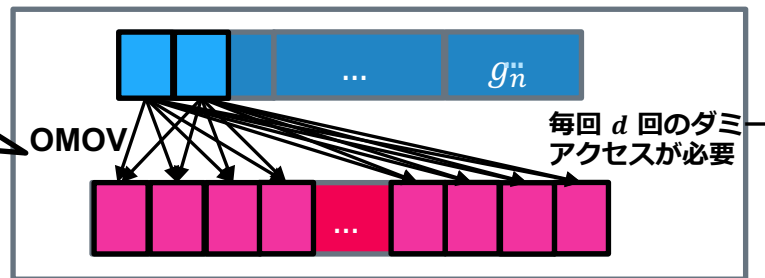
- Obliviousではない方法
 - $O(nk)$ で実行 (最速)
- ベースライン手法 (Oblivious)
 - 全体へのメモリアクセスに隠す
 - $O(nkd)$ (かなり遅い)
 - n : 参加者の数
 - k : スパース化パラメータの次元
 - d : 元の密なモデルパラメータの次元

プログラムの実行パスが入力データに非依存になるように、x64のCMOV命令 (2つのレジスタにデータをセットして条件フラグレジスタの値に従いどちらかの値を選択する) に基づくoblivious primitiveである**OMOV**を使用する必要がある
つまり、どこで本当にwriteしてるのかを隠す

Obliviousではない $O(nk)$



ベースライン手法 (oblivious) $O(nkd)$

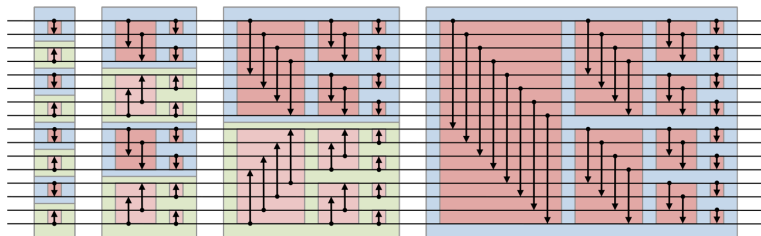


同様のことをもっと効率的に行うアルゴリズムを設計できないか？

より効率的なアルゴリズム

- Advanced method

- $O((nk + d) \log^2(nk + d))$ に減らせる
- Oblivious sort** を効果的に使って細かく設計する
 - Bitonic sortは下のソーティングネットワークに従いどんな入力に対しても全く同じ順番で値を比較してソートする



Bitonic sortのソーティングネットワーク
(画像: https://en.wikipedia.org/wiki/Bitonic_sorter)

n人から受け取ったスパースパラメータ (nk -次元)



Advanced method

$O((nk + d) \log^2(nk + d))$

詳細は次のページ



集約後のパラメータ (d -次元)

Advanced method の流れ

n 人から受け取ったスパースパラメータ (nk -次元) 下のようにゼロで初期化されたパラメータを用意する (d -次元)

$$\mathbf{G} \begin{bmatrix} g_1 & \dots & g_n \end{bmatrix} + \begin{bmatrix} (1, 0), (2, 0), (3, 0), \dots, (d-1, 0), (d, 0) \end{bmatrix}$$

⇒
初期化されたパラメータを結合する

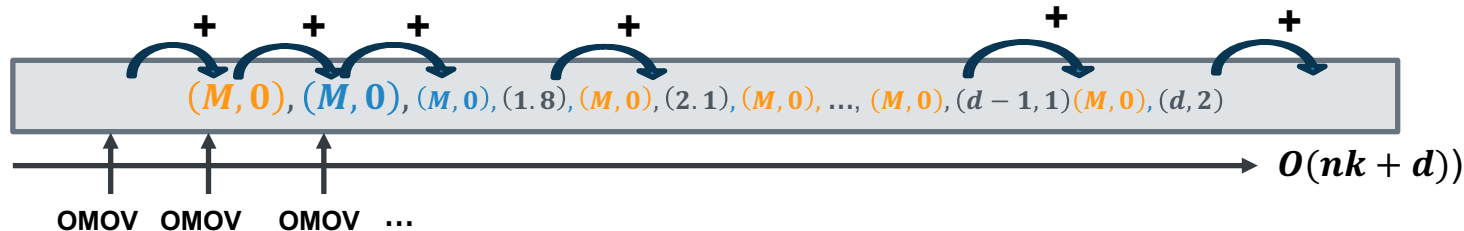
受け取ったパラメータと初期化パラメータがごちゃ混ぜになり、インデックス順にソートされる

⇒
Obliviousソート

$(1, 0), (1, 3), (1, 2), (1, 3), (2, 0), (2, 1), (3, 0), \dots, (d-1, 0), (d-1, 1), (d, 0), (d, 2)$

OMOVを実行しながら線形スキャンをして、インデックスごとに値を集約していく

⇒
Oblivious fold
(と名付けた処理)



最後にもう一度インデックスでソートすると

⇒
Obliviousソート

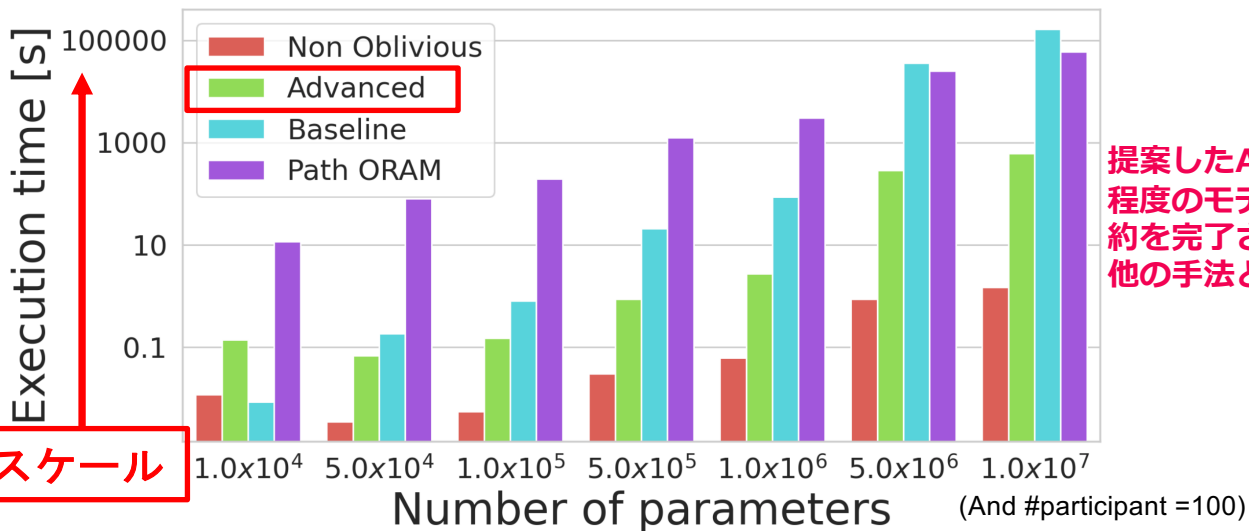
$(1, 8), (2, 1), \dots, (d-1, 1), (d, 2), (M, 0), (M, 0), \dots, (M, 0)$

最初の d -次元のベクトルが集約後のパラメータになっている

効率的である

PathORAMは、現在最も効率的なORAMの実装、ただしSGXのセキュリティモデル(レジスタのみがプライベートメモリ)に適応させる必要がある[NDSS18]

- 比較手法
 - Non Oblivious, Path ORAM (ZeroTrace [NDSS18]), Baseline, Advanced
- Intel SGXv1 (PRM: 128 MB)を使用



提案したAdvancedは100万次元程度のモデルであっても数秒で集約を完了させる
他の手法と比べても1桁以上早い

まとめと今後の課題

- サーバサイドTEEを使ったFLにおける攻撃と防御について研究しました
- パラメータにスパース化を適用した場合にプライバシーリスクがあることを示し、実際に攻撃によってプライバシー漏洩が起こることを実証しました
- その攻撃に対処するために、FLの集約操作に対して、効率の良いObliviousアルゴリズムを設計し評価しました
- 今後の課題
 - Malicious clientへの脅威は依然として防げない
 - 最近、Obliviousソートに対してBitonic sorting Networkの代わりにWaksman Network(フラグによって任意の並べ替えを表現できるスイッチングネットワーク)を使うことでさらに高速化できる可能性があることが示唆されたので[CCS23], 提案したFLの集約Obliviousアルゴリズムにも適用できる可能性がある
 - IntelSGXv2とv3の場合に性能特性が変わる可能性がある

[研究成果3]

Verifiable LDP

連合型分析における悪意のあるクライアントに対する防御手法

ターゲットとなるプライバシー・セキュリティ要件

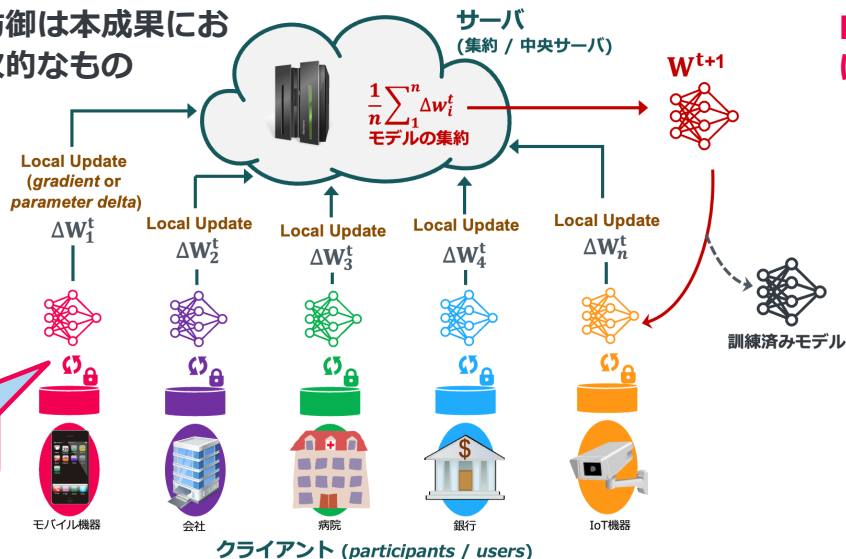
	A. Privacy against Central Server	B. Privacy against Client	C. Privacy against 3rd Party	D. Security against Central Server	E. Security against Client
VLDP [研究成果3] (FA+MPC)	(✓)	(✓)	(✓)	×	●

FLではなく Federated Analytics (FA) がターゲット

これらの防御は本成果においては副次的なもの

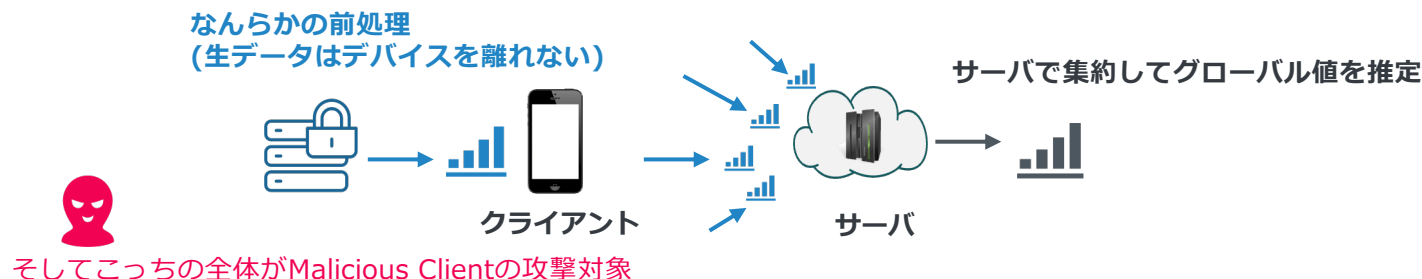
Malicious clientに対する防御は完全ではない

(5) 悪意のあるクライアントは Poisoning攻撃によってモデルの振る舞いをコントロール可能



Malicious Clientに対する防御は難しい

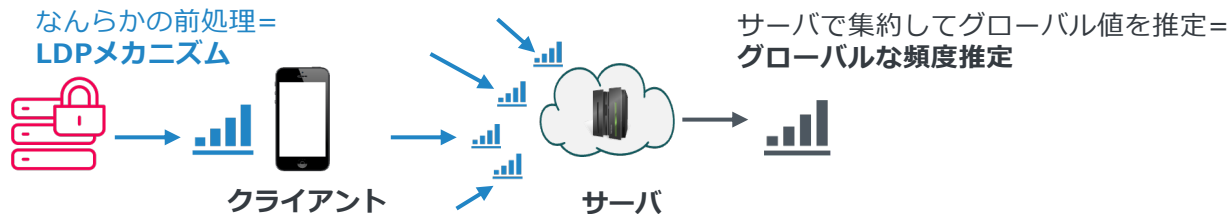
- 基本的にどんなタスクでもMalicious clientに対する防御(E)は非常に困難
 - 攻撃者は遠隔の端末で、ハードウェアは貧弱
 - 数が非常に多い可能性がある
 - クライアントはエッジでのプロセスを容易にコントロール可能
 - クライアントからの出力が“本物”かどうかを真に確かめるのはほぼ不可能、そもそもクライアントは嘘を入力するかもしれない
- FLにおけるMalicious clientに対する防御の初歩の研究として、FLではなくFederated Analytics (FA) に着目する
 - 多数のクライアントがローカルで生データを前処理 → サーバで集約しグローバル値を推定、というスキームは共通 (FLでは前処理がモデルの学習)



LDPを満たす頻度推定タスク

- LDPはサーバを信頼せずクライアントのローカル環境でデータをランダム化するプライバシーモデル
 - 実は、隣接データベースの定義が全く異なり、任意の2つの値 (※Central DPではレコードが最大1つ異なるデータベース) が隣接データベースとして定義
- LDPを満たす頻度推定 (Frequency Estimation) タスク
 - 各クライアント i ($i \in [N]$) はカテゴリ情報 $v_i \in [d]$ を持ち、何らかのLDPメカニズムを適用して $v_i \in [d]$ を $y_i \in [d]$ にランダム化してサーバに送信する
 - サーバは受けとった y_i ($i \in [N]$) から各カテゴリの頻度を推定する

FAとしての枠組み



代表的なLDPメカニズム

- 頻度推定のための代表的なLDPメカニズム **kRR** (k-Randomized Response) [NIPS14]
 - 他にも **OUE**, **OLH** [USENIX17] などがある
 - 有用性を最大化するビットエンコーディングなどの工夫

クライアント側でのランダム化
pは入力vがそのまま同じ値になる確率
qは別の値に変化する確率 (一様)

$$Pr[\mathcal{A}(v) = y] = \begin{cases} p = \frac{e^\epsilon}{e^\epsilon + d - 1}, & \text{if } y = v \\ q = \frac{1 - p}{d - 1} = \frac{1}{e^\epsilon + d - 1}, & \text{if } y \neq v \end{cases}$$

サーバ側のカテゴリkの頻度に対する不偏推定値
分母は (観測された数 - 嘘の数の期待値)

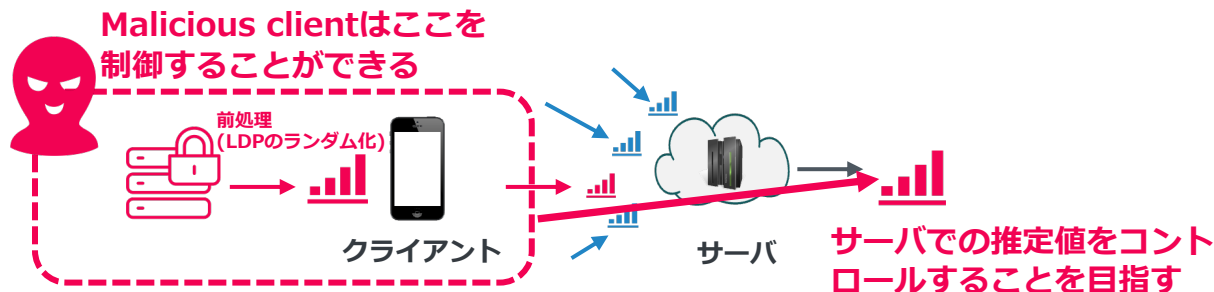
$$\mathcal{F}_k = \frac{\sum_i \mathbb{1}_{\text{Support}(y^i)}(k) - Nq}{p - q}$$

[NIPS14] Kairouz et al, Extremal mechanisms for local differential privacy. NIPS 2014.

[USENIX17] Wang et al, Locally differentially private protocols for frequency estimation. USENIX Security 2017.

Malicious Clientによる攻撃の可能性

- LDPを満たす頻度推定タスクにおいてもMalicious Clientによる攻撃が提案されている [USENIX21] [SP21]
 - [USENIX21] Targeted Attack
 - 標的のアイテムの頻度を不正に大きくすることを目指す
 - [SP21] Untargeted Attack
 - 推定値の精度を真の値から不当に乖離させることを目指す
- 直感的にはLDPのランダム化を利用して、非プライベートな場合よりも攻撃にレバレッジを効かせることが可能、なのでプライバシーが強いほど攻撃も強い



[USENIX21] Cao et al, Data poisoning attacks to local differential privacy protocols. USENIX Security 2021.

[SP21] Cheu et al, Manipulation Attacks in Local Differential Privacy. S&P 2021.

Malicious Clientによる攻撃の観察

- Malicious Clientによる攻撃には以下のレベルがある
 - 実は、同様のクラス分けがFLにも知られており、**Data poisoning attack** と **Model update poisoning** と呼ばれる
 - そして同様に、Model update poisoning > Data poisoning attack が経験的に示唆されている [ICML19]

攻撃なし

真の入力を選択し、ランダム化メカニズムを正しく適用し、その出力をサーバに共有する

$v \in [d]$
True input

ランダム化
メカニズム ϕ

$\phi(v)$

Server

Input-manipulation (Data poisoning attack)

偽の入力 v' を選択し、ランダム化メカニズムを正しく適用し、その出力をサーバに共有する

$v' \in [d]$
Fake input

ランダム化
メカニズム ϕ

$\phi(v')$

Server

Output-manipulation (model update poisoning)

偽の出力 v' を作成し、その出力を直接サーバに共有する

$v' \in [d]$
Fake output

ランダム化なしで
直接送信する

v'

Server

攻撃強い

Input-manipulation
を一般的に防ぐのは非
常に困難である

2つの攻撃の理論的な分離

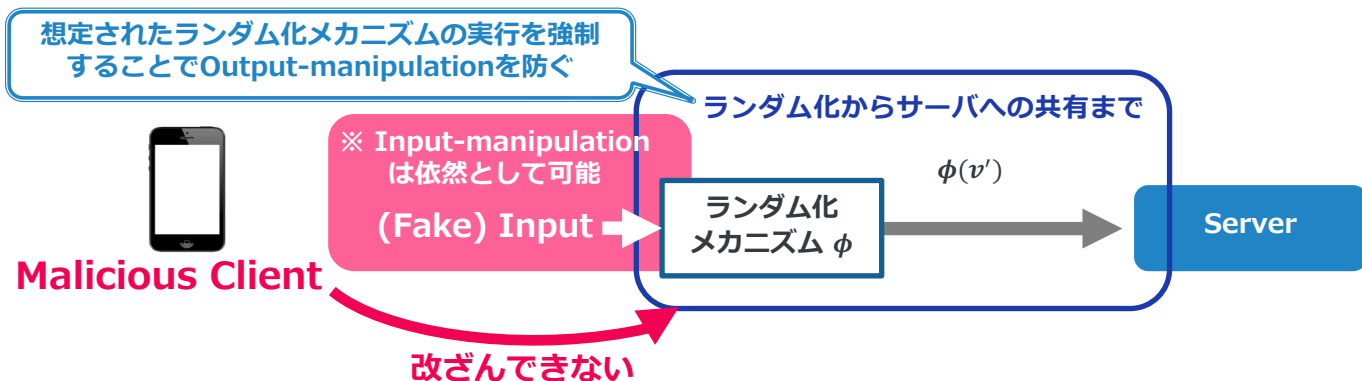
- Input-/Output-manipulationは理論的分離が可能
 - Output-manipulation** と **Input-manipulation** の間のLDP頻度推定タスクにおけるL1距離の期待利得の差を以下のように計算可能 (FLではまだOpen)
 - ちなみに, プライバシバジェットに反比例して大きくなる

Targeted Attack	kRR	OUE	OLH
[USENIX21]	$+\left(\frac{\beta(d-r)}{e^\epsilon-1}\right)$	$+\left(\beta(2r-1)+\frac{2\beta r}{e^\epsilon-1}\right)$	$+\left(\beta(2r-1)+\frac{2\beta r}{e^\epsilon-1}\right)$
Untargeted Attack [SP21]	$\times \Omega\left(\frac{\sqrt{d}}{\epsilon}\right)$		

- よって, **Output-manipulationのみを防御することを目的にする**
 - Input-manipulation は諦める, Blockchain(入力データの検証) やincentive設計 (正しく貢献しないと意味がない) などが重要な可能性がある

防御のアイデア

- 集約サーバから**検証可能なランダム化メカニズム**を使ってOutput-manipulation を完全に防ぐ
 - 集約サーバは、クライアント側から得られた出力が、想定されたランダム化メカニズムの出力であるかどうかを外部から検証する
 - Malicious Clientは、入力データを1つ選択する、という行動以外とれない
- クライアント側に、Remote Attestationが可能なTEEを使う、というのは強すぎる仮定なのでソフトウェアベースの方法が必要**



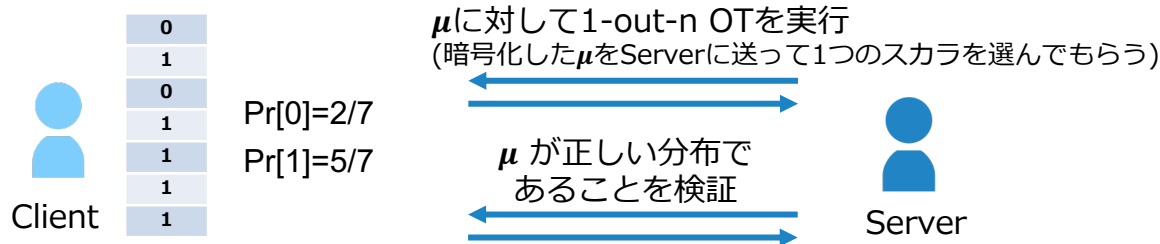
もし何らかの不正(想定されたプロトコルからの逸脱)があった場合、必ず失敗する

提案手法

- Cryptographic Randomized Response Techniques [PKC04] (CRRT) を拡張して, LDPの頻度推定プロトコルにおけるランダム化メカニズムを検証可能にする
 - **kRR, OUE, OLH** の3手法に対して設計する

手法の概要

ランダムサンプリングを Oblivious Transfer(OT) によってサーバとクライアントの二者間でシミュレートし, OTの対象となるLDPメカニズムの確率分布を表すデータ構造の分布の正しさをPedersenのコミットメントスキームの加法的準同型性によって検証する

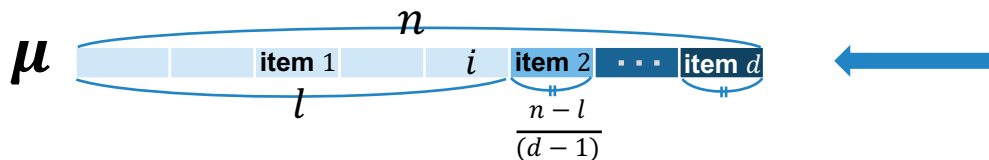


最初にLDPメカニズムで使用するカテゴリカル分布を表すベクトルを用意: μ

TEEやTrusted Setupなど特別な仮定は不要!

分布ベクトルを用意する

- 最初にLDPメカニズムで使用するカテゴリカル分布を表すベクトル μ を用意する



→ この μ から一様にアイテムをサンプリングできれば、アイテム i の選択確率が以下となり、 ϵ -LDPを満たす

$$\begin{cases} \Pr[i] = \frac{l}{n} \left(\leq \frac{e^\epsilon}{d-1 + e^\epsilon} = p \right) & \text{if } i \text{ is selected item} \\ \Pr[i] = \frac{n-l}{n(d-1)} \left(\geq \frac{1}{d-1 + e^\epsilon} = q \right) & \text{otherwise} \end{cases}$$

この分布ベクトル μ は離散的な値しか表現できないのでLDPの満たす分布を全て完全に表現できるわけではない、ベクトルの長さのパラメータ ($width$) を大きくすると誤差は小さくなる(右図)、しかし計算コストは増加する

Algorithm 1 DECIDESHAREDPARAMETERS

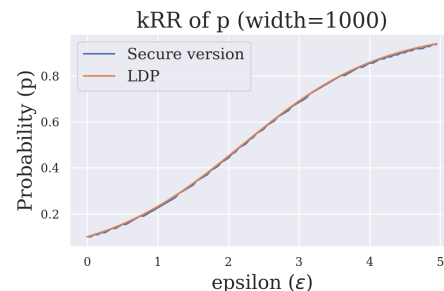
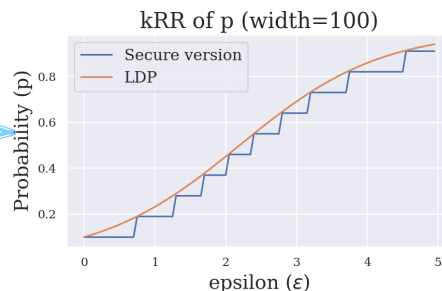
Input: $\epsilon, width$

```

1:  $i \leftarrow \lfloor \frac{e^\epsilon}{(d-1) + e^\epsilon} \rfloor$  // as an integer
2: while  $i > 0$  do
3:   if  $(width - i)$  divides  $(d - 1)$  then
4:      $g \leftarrow \gcd(i, width, \frac{width-i}{d-1})$ 
5:      $l, n \leftarrow \frac{i}{g}, \frac{width}{g}$ 
6:     break
7:   end if
8:    $i \leftarrow i - 1$ 
9: end while
10:  $z \leftarrow \max([l, \frac{n-l}{d-1}]) + 1$ 

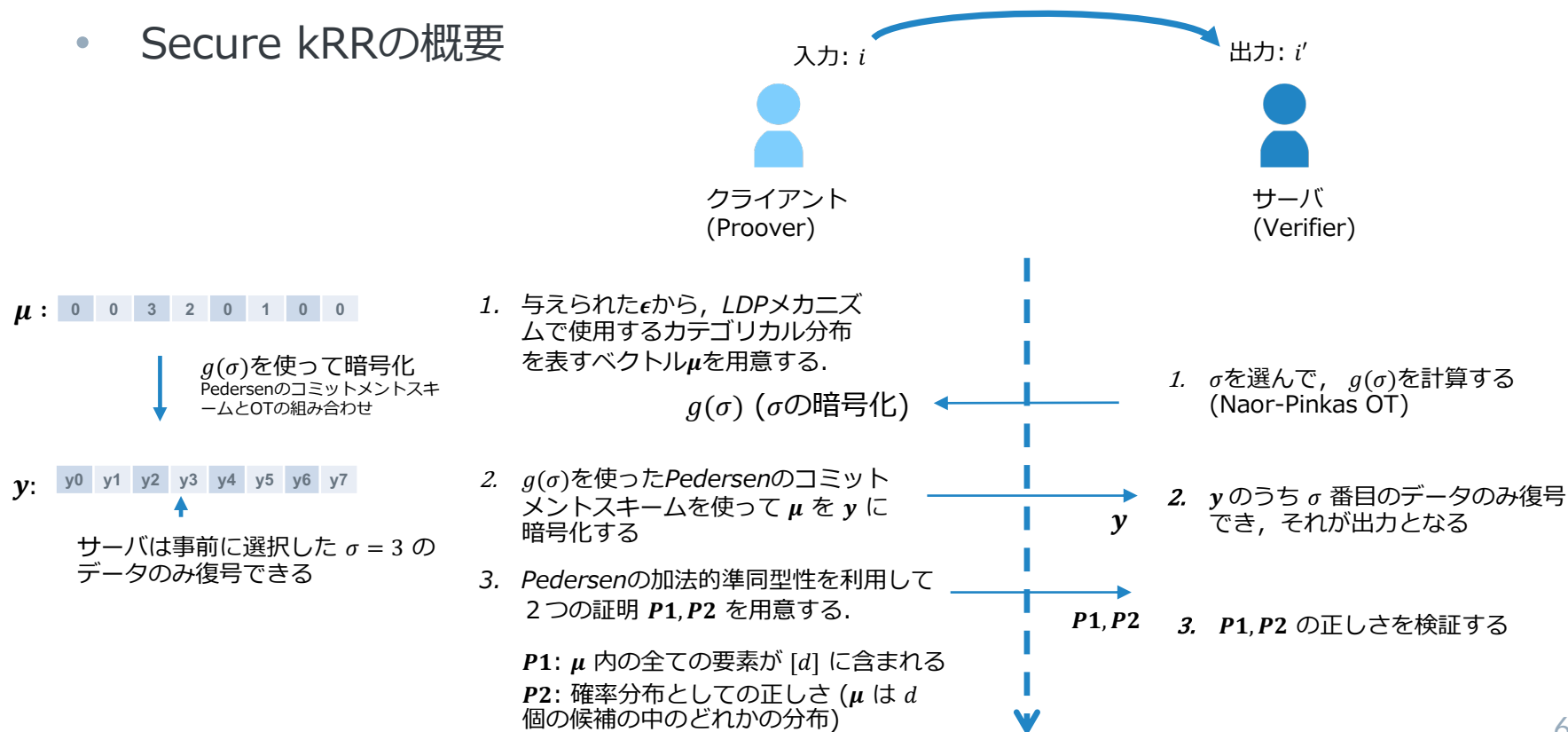
```

Output: l, n, z



検証可能な Secure kRR

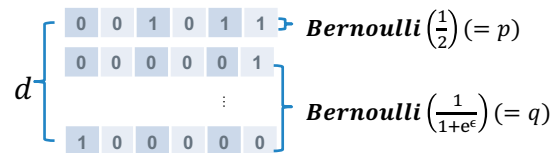
- Secure kRRの概要



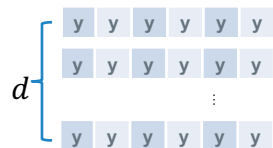
検証可能な Secure OUE

- Secure OUEの概要
 - Secure OLHも大まかに類似

$\vec{\mu}$ (ベクトルはd個, OUEの出力分布に従う)



$\vec{y}$ $g(\sigma)$ を使って暗号化



- 与えられた ϵ から, LDPメカニズムで使用するベルヌーイ分布を表すベクトル $\vec{\mu}$ を用意する.
- $Pedersen$ のコミットメントスキームと $g(\sigma)$ を使って $\vec{\mu}$ を $\vec{y}$ に暗号化
- $Pedersen$ の加法的準同型性を利用した3つの証明 $P1, P2, P3$ を用意する

$P1$: μ_i の全ての要素がビット(0か1)

$P2$: 全ての $\mu_i \in \vec{\mu}$ が p or q のどちらかの分布

$P3$: $\mu_i \in \vec{\mu}$ のうち p の数は 1 で q の数は $d - 1$

入力: i

クライアント
(Prover)

OUEでは出力はd次元
のビットベクトル

出力: $[0, 1, 0, 1, 0, \dots, 0]$

サーバ
(Verifier)

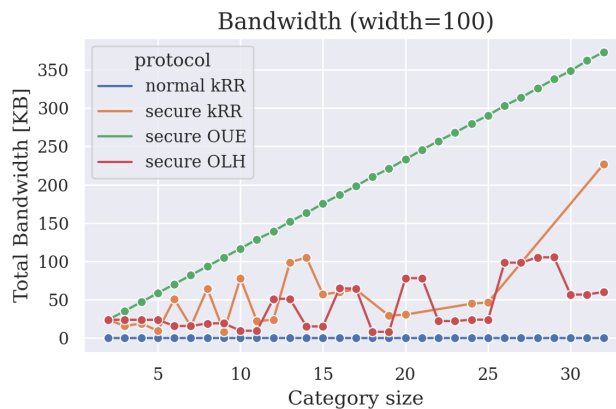
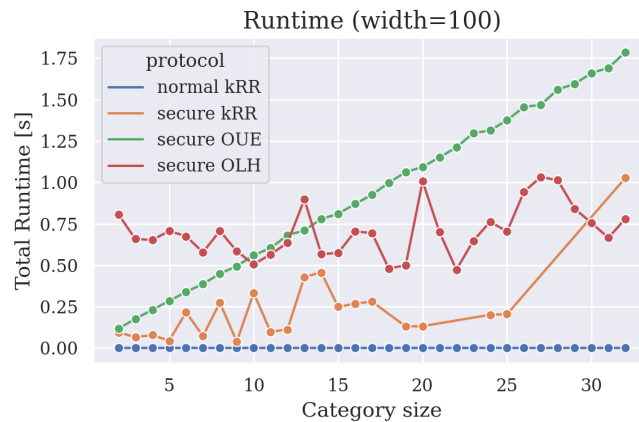
- ベクトル $\sigma = (\sigma_1, \dots, \sigma_n)$ を選び, $g(\sigma)$ を計算する (multivariate Naor-Pinkas OT)
- $y_i \in \vec{y}$ のうち σ_i 番目のデータのみ復号でき, それが出力となる ($i \in [d]$) で d 次元の出力が得られる)
- $P1, P2, P3$ の正しさを検証する

満たすべきセキュリティ条件

- Indistinguishability
 - 手法内で使っているPedersenのコミットメントスキームとNaor-Pinkas OTを組み合わせた暗号スキームは *left-or-right secure* であることが示されている[PKC04]のでOK
- Verifiability
 - *ZK disjunctive proof* ($P1, P2$) [CRYPTO94] からOK
 - *Discrete logarithm problem* ($P3$)の難しさからOK
- Local Differential Privacy
 - μ の確率分布はLDPの確率分布が上界になっているのでOK

効率性の評価

- クライアントは1つのアイテムを1.0-LDPでサーバに送信
- 実行時間とメッセージサイズ
 - それぞれ、現実的なスケールで実行可能
 - 証明などに1秒くらいはかかる
 - メッセージサイズはそれなりに大きくなる
 - カテゴリーサイズの増加でさらに大きくなる



横軸はカテゴリーのサイズ

FLへの一般化可能性

- Output/Input-manipulationの議論はFLでも同様に成立するため、私が提案している防御スキーム、クライアント側の前処理プロセス(すなわちモデルの学習)の検証、は**依然として有効な防御**と言える
- FAとFLの違いはローカルの前処理プロセスの複雑さ
 - FLのローカル前処理プロセスは複雑な最適化処理を含み、本提案手法を直接適用するのは技術的な観点で難しい
- 検証可能なLDPメカニズムについて、ZKP (ZK-SNARK) を使った拡張が提案されている [ARES22]
 - ZK-SNARKは任意の算術回路に対する検証性を提供することができる
 - [Arxiv23] ではZK-SNARKを使ってクライアントサイドでの学習の検証を提案しているが、スケーラビリティに乏しい (Irisなどでギリギリくらい)
 - ZK-SNARKはそもそもTrusted Setupなど強い仮定が多い
- ZKP自体を進化させるか、私の提案手法のように軽いMPC技術を使ってFLの検証性を(例えば、部分的に)提供する方法、が次なる方向性と言える

[ARES22] Garrido et al, Towards Verifiable Differentially-Private Polling. ARES 2022.

[Arxiv23] Xiong et al, Zero-Knowledge Proof-based Practical Federated Learning on Blockchain. Arxiv preprint 2023.

まとめと今後の課題

- 典型的なFederated Analyticsタスクである, LDPの頻度推定に注目し, Output-manipulation を防ぐことの重要性を示しました
- 検証可能なLDPプロトコルを提案して, Output-manipulation を完全に防ぐ手法を示しました
- 今後の課題
 - Input-manipulationに対する防御
 - FLへの拡張

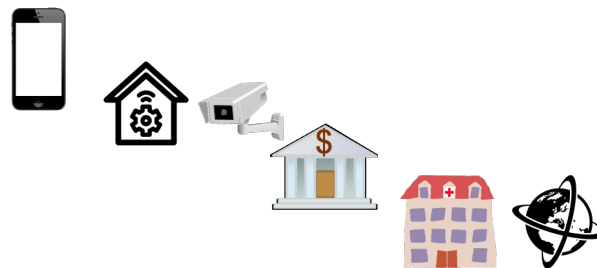
まとめ

本博士論文の社会へのインパクト

- 現在のプライバシー保護技術(FLを含む)は「試験段階」や「企業のアピール」の域を出ず本番運用に至らない, 主な原因は以下
 - (限界1) プライバシ保証に明らかな不足がある
 - 使用者側がプライバシーを強く主張できないのが問題!
 - (限界2) プライバシ保護技術の導入によって付随するリスクがある
 - 守りの技術への投資は難しいのにリスクまであったらそれは厳しい!
- 本博論の成果は**一般的なFL(インパクト大)に対する**上記2点を解決し, FLが「本番レベル」で使われることを促進することが期待される
 - ULDP-FL → DPの提供する保証の明らかな欠点を克服 (1)
 - OLIVE → TEEの提供する保証の明らかな欠点を克服 (1)
 - VLDP → プライバシ保護技術の導入によって逆に起こるセキュリティリスクを防ぐ (2)

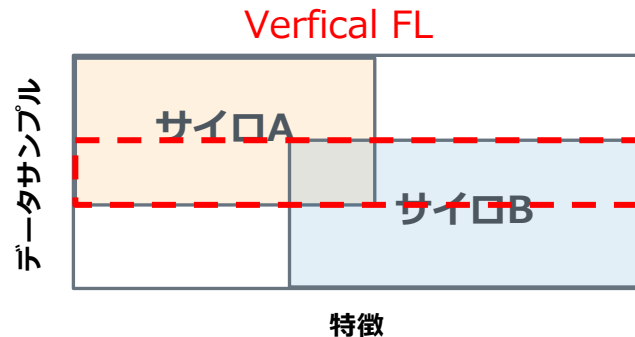
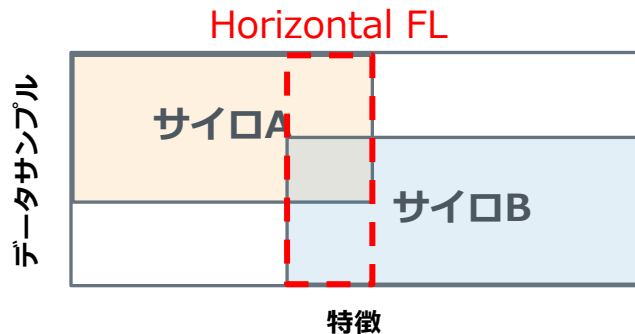
実世界における適用先はとても広い

- 3つの提案手法の応用先はほとんど全てのFLアプリケーション
 - 以下のような、FLが検討され始めている分野一般に対して必要に応じて適用可能
 - モバイルデバイス
 - スマートホーム/IoTデバイス
 - 産業（会社，銀行間など）
 - 医療（病院間，国際間など）
- それぞれの提案手法は独自のトレードオフを持っているので，必要に応じて使われるべきだが，実際「本番レベル」では必要になることが多いのではないかと



本博士論文の限界

- FLのプライバシー・セキュリティの課題は、**A-E(P.7)**だけではない
 - 分散システムが故に学習データのオプトアウトや追跡性が困難である, など
- ターゲットはHorizontal FLだけ
 - FLには他にもVertical FLやSplit Learningなどの変種があって注目されているが, プライバシやセキュリティのリスクは全く異なる可能性がある



全体の結論

- FLにおける PrivacyとSecurityの課題を5つに整理しました
- それらの5つの課題を克服するためにTEE, DPといった高度なプライバシー保護技術をFLに統合するための3つのフレームワークについて研究し, 本分野に大きく貢献しました
 - クロスサイロFLにおいて, User-level DPをサイロを跨いで保証するための手法を提案しました
 - サーバサイドTEEを使ったFLにおいて, 漏洩しうるメモリアクセスパターンから攻撃が可能であることを示し, それに対する防御を提案しました
 - 単純なFAタスクである, LDPを満たした頻度推定において, Malicious Clientに対する防御のためのクライアント側の前処理プロセスの検証手法を提案しました
- サーバサイドTEEとDPをFLに統合することは(私の研究の貢献もあり)成熟しつつあるため, 特に未知な分野はMalicious Clientに対する防御ではないかと思えます(特に, Input-manipulation (Data poisoning) は危険な割にかなりOpenではないかと思えます)

本論文の内容に関連する発表文献

- ジャーナル論文

- [研究成果2] Fumiyuki Kato, Yang Cao and Masatoshi Yoshikawa. OLIVE: Oblivious Federated Learning on Trusted Execution Environ- ment against the risk of sparsification. Proc. **VLDB** Endow. 16, 10 (June 2023), 2404–2417.
- [研究成果A, 付録] Fumiyuki Kato, Tsubasa Takahashi, Shun Takagi, Yang Cao, Seng Pei Liew, and Masatoshi Yoshikawa. HDPView: differentially private materialized view for exploring high dimensional relational data. Proc. **VLDB** Endow. 15, 9 (May 2022), 1766–1778.
- [研究成果B, 付録] Fumiyuki Kato, Yang Cao and Masatoshi Yoshikawa. PCT-TEE: Trajectory-based Private Contact Tracing System with Trusted Execution Environment. **ACM Trans. Spatial Algorithms** Syst. 8, 2, Article 13 (June 2022), 35 pages.

- 国際会議論文

- [研究成果1] Fumiyuki Kato, Li Xiong, Shun Takagi, Yang Cao, Masatoshi Yoshikawa. ULDP-FL: Federated Learning with Across Silo User-Level Differential Privacy. Submitted to 50th International Conference on Very Large Databases (**VLDB 2024**), **Under Review**. arXiv preprint arXiv:2308.12210 (2023).
- [研究成果3] Fumiyuki Kato, Yang Cao and Masatoshi Yoshikawa. Preventing manipulation attack in local differential privacy using veri- fiable randomization mechanism. Data and Applications Security and Privacy XXXV: 35th Annual IFIP WG 11.3 Conference (**DBSec**), DB- Sec 2021, Calgary, Canada, July 19–20, 2021, Proceedings 35. Springer International Publishing, 2021.
- [研究成果B, 付録] Fumiyuki Kato, Yang Cao and Masatoshi Yoshikawa. Secure and efficient trajectory-based contact tracing using trusted hard- ware. In 2020 IEEE International Conference on Big Data (**Big Data**) (pp. 4016-4025). IEEE. 7th International Workshop on Privacy and Security of Big Data.

[付録]

議論: 3つの提案フレームワークの関係性

- 提案した3つのフレームワークは組み合わせることができる
 - サーバサイドTEEの利用, User-level DPを保証するアルゴリズム, クライアント側の前処理プロセスの検証, はそれぞれ全く依存していない
 - これによって(**A-E**)の要件を全てカバーすることができる
- 一方で, それぞれのフレームワークは独自のトレードオフを持つ
 - User-level DP → 有用性とプライバシー
 - サーバサイドTEE → ハードウェア要件とプライバシー・セキュリティ
 - クライアント側検証性 → セキュリティと計算コスト
- ユースケース例
 - エンドユーザ向けのアプリ上で動くレコメンドモデル: **A,B,C,D,E** が重要
 - 病院間の連携: **A,B,D** のみが重要

ULDP-FLに関する付録

DP-FLの中での位置付け

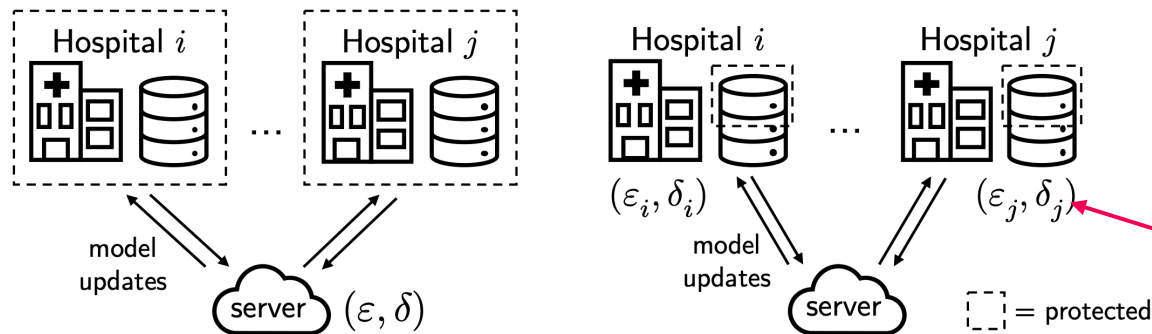
			Record-level	User-level
Centralized ML (CML)			[DPSGD16]	[BigData20] [NIPS21] [PETS20] [ICML19], [ISIT22] (user-level LDP?) ,[VLDB20] (<i>Across multi service-LDP</i>)
FL	Cross-device		(same as CML)	[NIPS17W] [ICML18] [ICML21](shuffleDP) [ICML21+] 各デバイスが一人のユーザのデータを全て持つという仮定が成立する
	Cross-silo	Silo-specific DP それぞれのサイロ内で 独立してDPを保証する ことを目指す	[VLDB21] [NIPS22] [ICML23] [AISTATS23]	(trivial) user-level DPを満たすCMLの手 法がそのまま使えるが [NIPS22][ICML23][AISTATS23]などは Group DPを使うと言及しているだけ
		(Across silo) DP 全てのサイロがグロー バルに1つのDPを保証 することを目指す	[PMLR21] (CML と同じ)	私の研究はここ！ 一人のユーザが一つのサイロにしか続かないと仮定できない

前の表の参考文献

- [DPSGD16] Deep Learning with Differential Privacy
- [BigData20] Broadening Differential Privacy for Deep Learning Against Model Inversion Attacks
- [NIPS21] Learning with User-Level Privacy
- [PETS20] Differentially Private SQL with Bounded User Contribution
- [ICML19] Bounding user contributions: A bias-variance trade-off in differential privacy.
- [NIPS22] On Privacy and Personalization in Cross-Silo Federated Learning
- [ICML23] Anonymous Private Federated Learning Without a Trusted Server: Optimal Algorithms for Convex Losses (under review)
- [ICML21] Shuffled Model of Differential Privacy in Federated Learning
- [NIPS17W] Differentially private federated learning: A client level perspective
- [ICML18] Learning differentially private recurrent language models
- [ICML21+] The distributed discrete gaussian mechanism for federated learning with secure aggregation
- [VLDB21] Projected Federated Averaging with Heterogeneous Differential Privacy
- [VLDB20] Collecting and analyzing data jointly from multiple services under local differential privacy
- [PMLR21] The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation.
- [AISTATS23] Private Non-Convex Federated Learning Without a Trusted Server

最近のクロスサイロDP-FL

- Silo-specific sample-level DP [NIPS22] (サイロを跨いでいない)
 - 別名 Inter-silo record-level DP [AISTATS23][ICLR23]
 - これは、各サイロの中で独立してRecord-level DPを保証する方法



サイロごとに異なるプライバシーバジェットを持つサイロ間のデータベースは完全に独立しているという仮定

Client-level DP: Participating silos are protected (with notions of local/central/distributed DP)

Silo-specific sample-level DP: Individual records within silos are protected with silo-specific targets

Figure 1: **Client-level DP** vs **Silo-specific sample-level DP**.

最近のクロスサイロDP-FL

- [NIPS22][AISTATS23][ICLR23] はユーザレベルのDPを達成するために、彼らの手法がGroup DPを使って拡張できると言及するにとどまっている
 - 私の研究では、ULDP-GROUP- k をベースラインとして設計している
- サイロを跨いだ設定におけるGroup DPの問題点は以下に要約される
 - **(1) super linear privacy bound degradation with k**
 - (2) hard to determine optimal group size k
 - (3) need to remove data across silo to limit the maximum number of records to k

Distributed DPについての注意

- ▷ Distributed DP (DDP) とは、信頼できないサーバなしのCentral DP
 - 例えば、クロスサイロFLでは、サイロはサーバを信頼したくないがCentral DPを達成したい
 - しかしながら、このサーバへの信頼の制約は実は厳密ではないことが多い
 - 例えば、[ICML21]や多くの研究はDDPを達成するためにSecure Aggregationを使っているが、参加しているdeviceの総数を知っているし、各ラウンドに参加するサンプリングされたユーザも知っている必要がある。これはサーバに対する厳密なDPの保証に影響する可能性がある
 - これらは、外部のモデルユーザに対して最終的に公開される訓練済みモデルに対するDPの保証には関係ない[NIPS21]

プロトコルの流れ (再喝)

1. (サーバと各サイロ) DH鍵交換を通じて、サーバ経由で各サイロペアごとに共有鍵を作成する、サーバはPaillierの公開鍵をサイロに配布する
2. (各サイロ) 共有鍵を共通シードに、共通の乱数列を作成する
3. (各サイロ) 2で得られた乱数列を使ってユーザのレコード数のヒストグラムに対してMultiplicative Blindingを行う
4. (サーバと各サイロ) Secure Aggregationを使ってBlindedヒストグラムを集約
5. (サーバ) Blindedヒストグラムに対して逆数を計算して、(さらに必要であれば、ユーザレベルのサブサンプリングをして重みをゼロ化する)値をPaillier Encryptionによって暗号化する
6. (サイロ) ローカルの学習中にPaillierで暗号化された重みで重み付ける
7. (サーバと各サイロ) Secure Aggregationによってモデルパラメータを集約する
8. (サーバ) 集約されたデータをPaillierの秘密鍵を使って復号してグローバルモデルを更新する
9. (5,6,7,8)を繰り返す

Protocol (1)

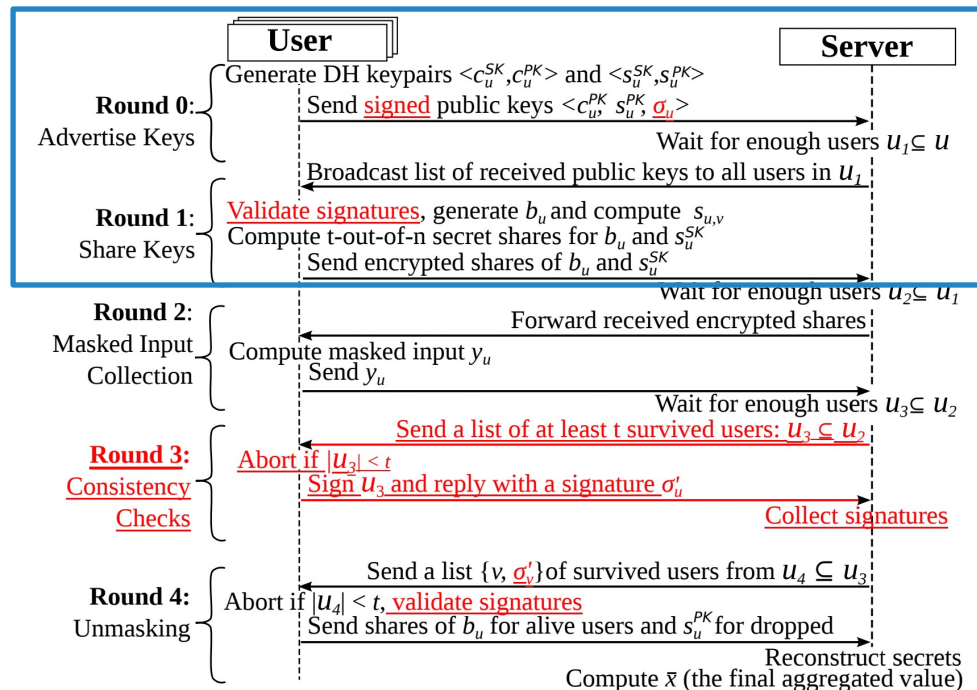
1.(サーバと各サイロ) DH鍵交換を通じて、サーバ経由で各サイロペアごとに共有鍵を作成する。この共有鍵から全体の共通ランダムシードを生成する。(一つのサイロがリーダーとなりランダムシードを作成し、他のサイロに、DH共有鍵を使って暗号化して配布すればよい)

また、サーバはPaillierの公開鍵をサイロに配布する。これはモジュラス n を含む。以降の暗号空間のための有限体 $\mathbb{F}_n$ に使われる。

クロスデバイスFLではデバイスのドロップアウトが重要なので秘密分散を組み合わせることが重要 [CCS17], 一方でクロスサイロFLでは秘密分散が必要ない

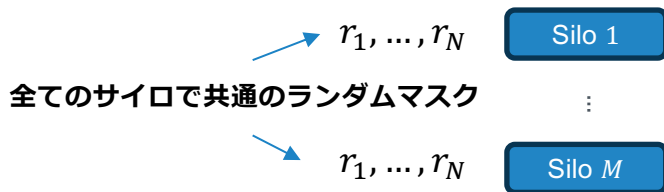
[CCS17] のSecure Aggregationの図

私のプロトコルも同様にDH鍵交換と、その共有鍵を使った共通の加法マスクの生成を行っている。青枠はDH鍵交換。赤文字は、malicious clientのための拡張

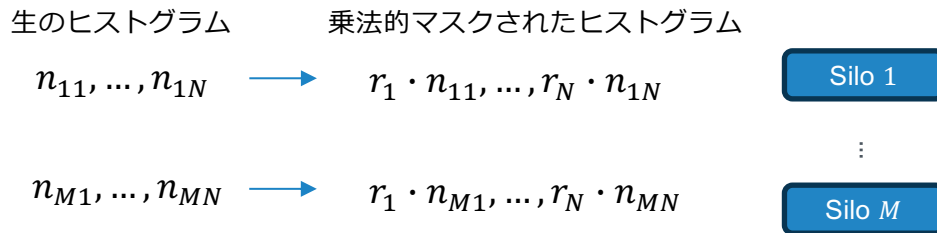


Protocol (2)(3)

2. (各サイロ) ステップ1で得られた共通のランダムシードを使って, 共通のランダムマスク r_u ($\forall u \in [N]$) を作成する. $r_u \in \mathbb{F}_n$ である.

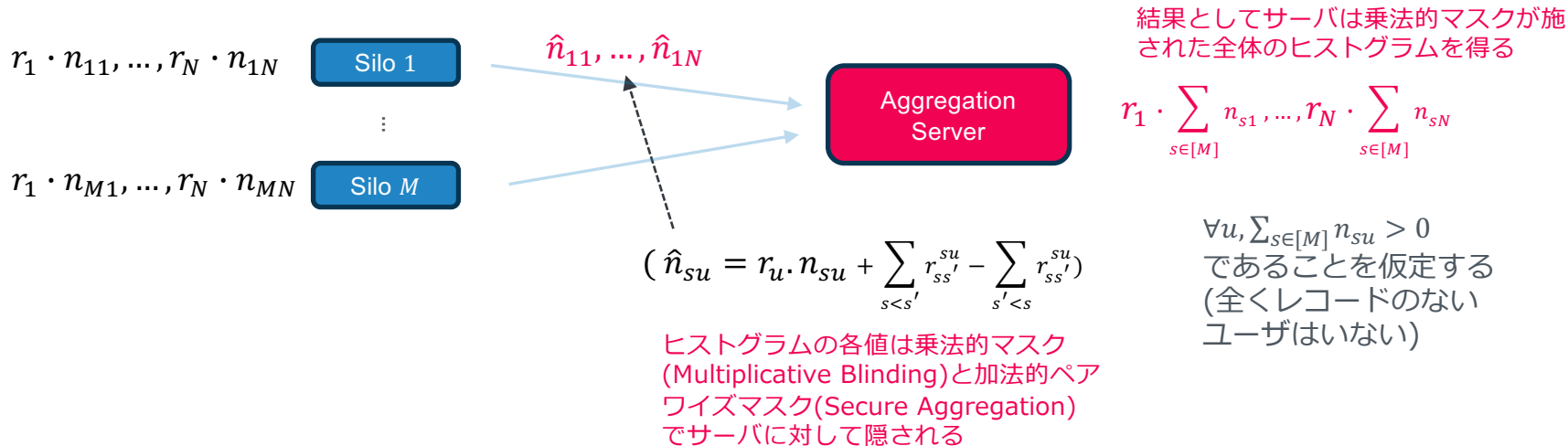


3. (各サイロ) 各サイロが持つユーザのレコード数のヒストグラムに対して Multiplicative Blinding (乗法的ランダムマスクを使った匿名化)を適用する



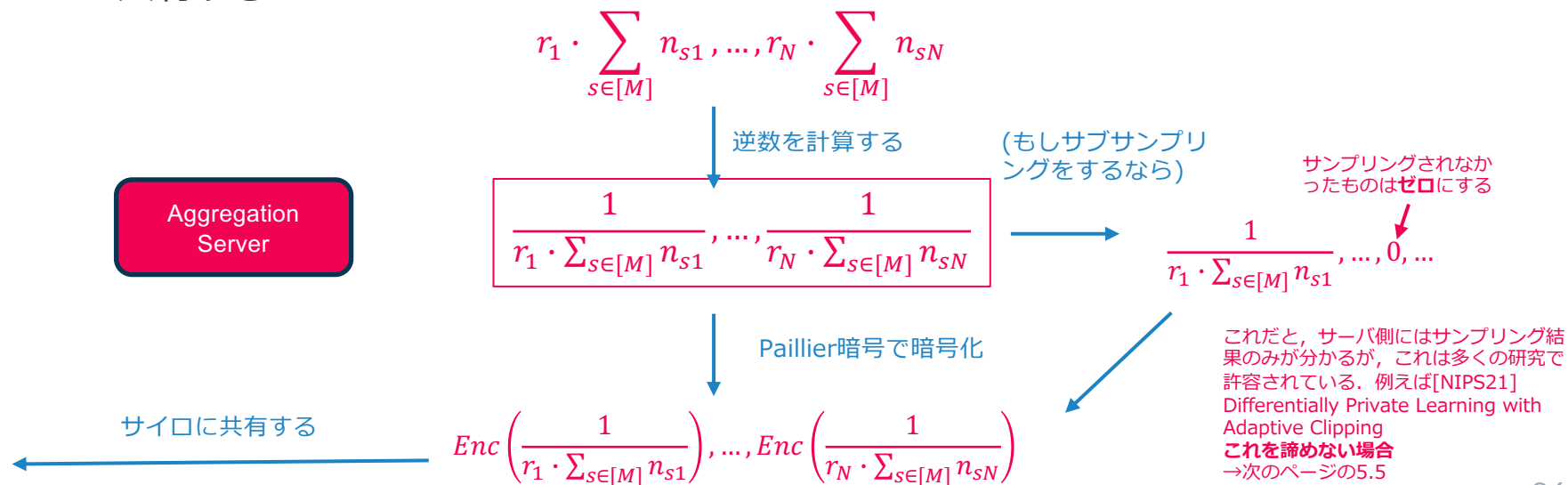
Protocol (4)

4. (サーバと各サイロ) マスクされたヒストグラム($\hat{n}_{11}, \dots, \hat{n}_{1N} = r_1 \cdot n_{11}, \dots, r_N \cdot n_{1N}$) に対してSecure Aggregationを実行する。(サーバはマスクされた集約結果にのみアクセスできることが保証される。なければ比などが漏れる。) Secure Aggregationは, サイロペアワイズな加法的マスク $r_{ss'}^{su} \in \mathbb{F}_n$ を生成して行う



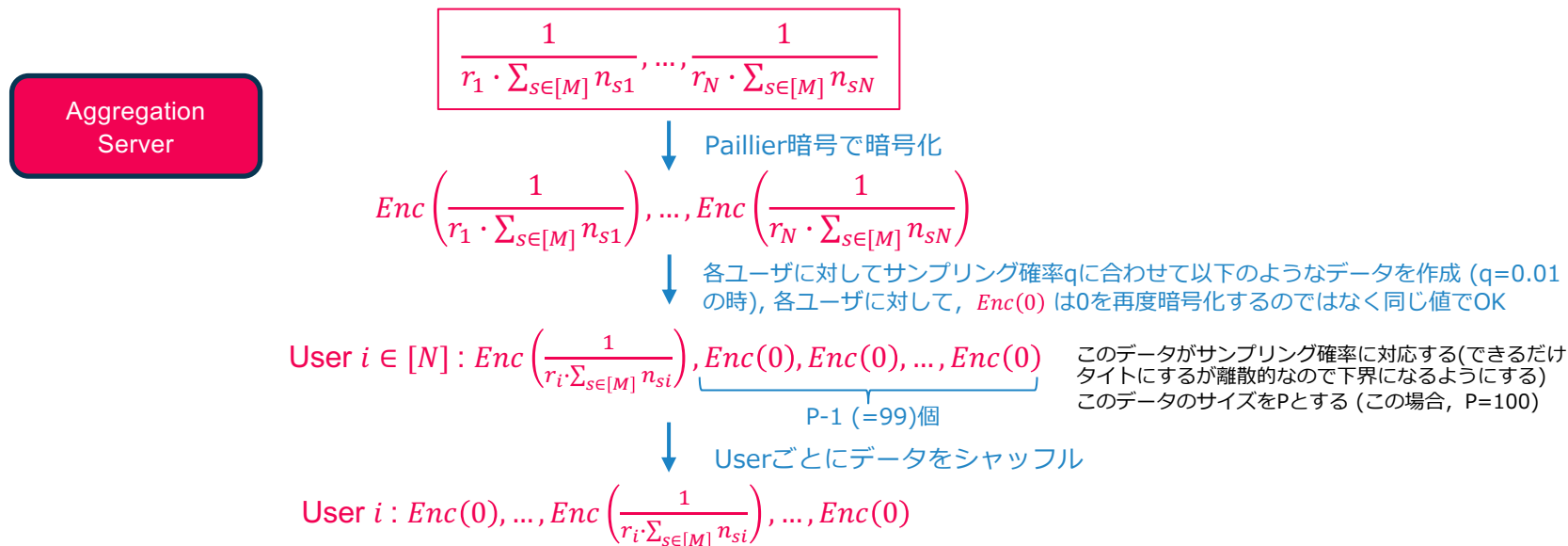
Protocol (5)

5. (サーバ) 乗法的マスクされたヒストグラム($r_1 \cdot \sum_{s \in [M]} n_{s1}, \dots, r_N \cdot \sum_{s \in [M]} n_{sN}$)に対して逆数(有限体 $\mathbb{F}_n$ 上の乗法逆元, 拡張ユークリッド互除法で高速に計算可能)を計算する($\frac{1}{r_1 \cdot \sum_{s \in [M]} n_{s1}}, \dots, \frac{1}{r_N \cdot \sum_{s \in [M]} n_{sN}}$). サブサンプリングをするなら, 選択されなかったユーザに対応する値をゼロ化する. Paillier暗号の秘密鍵を使って暗号化して, サイロに共有する.



Protocol (5.5)

5.5.1. 1-out-of-P Oblivious Transfer ([Naor01] or [Evan85])を使ったプライベートなユーザレベルのサブサンプリングの実装. 最初に, サーバ側で以下のようなデータを準備する (OT部分は, [研究成果3]や [张艳硕20] でも使っているアイデア)



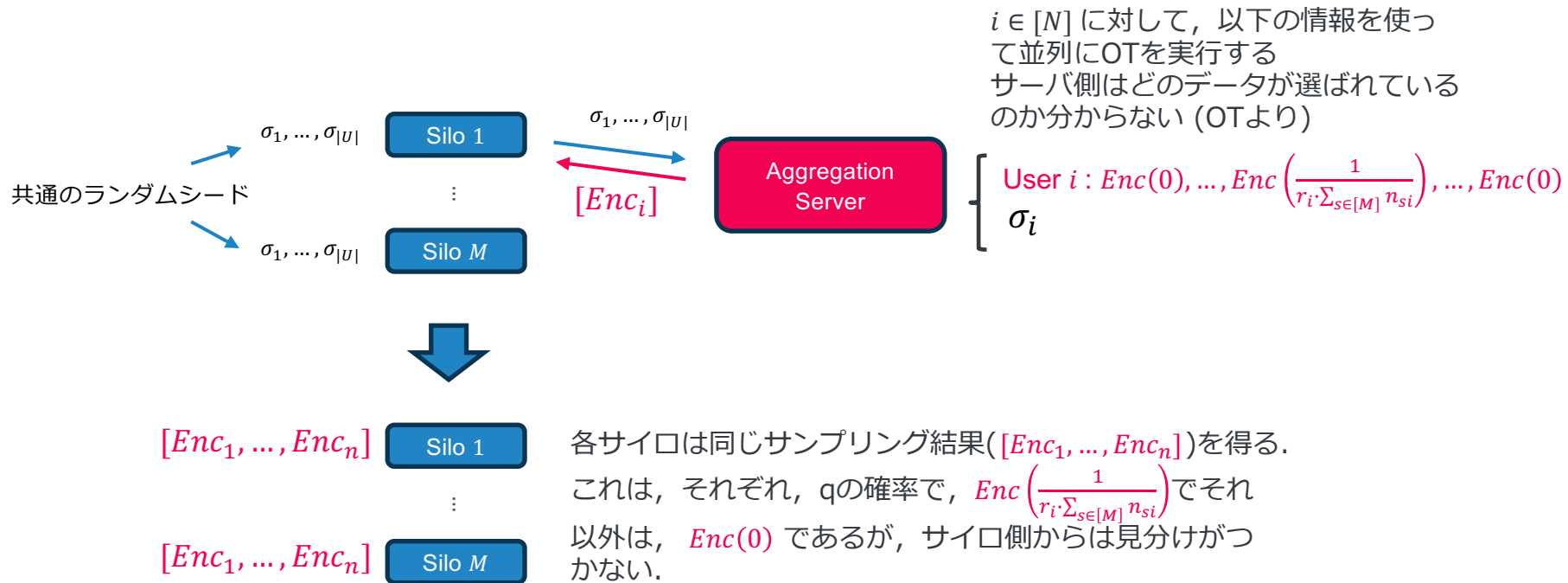
[Naor01] Naor, Efficient oblivious transfer protocols. SODA 2001.

[Evan85] Evan, A randomized protocol for signing contracts.

[张艳硕20] 张艳硕, 概率型 2 选 1 不经意传输协议的方案设计. Journal of Cryptologic Research, 2021 (中国語)

Protocol (5.5)

5.5.2. 各サイロは、共通のシードを使って、1~Pまでの数字をランダムに $|U|$ (ユーザ数)個選んでおく ($\sigma_1, \dots, \sigma_{|U|} \in P$)



Protocol (6)

6. (サイロ) ローカルでの学習中, Paillierで暗号化されて共有されている重み $(Enc(\frac{1}{r_1 \cdot \sum_{s \in [M]} n_{s1}}), \dots, Enc(\frac{1}{r_N \cdot \sum_{s \in [M]} n_{sN}}))$ を使ってパラメータを重み付けする(単に有限体上 $\mathbb{F}_n$ での乗算を計算する).

Silo s

各サイロ s 内で, それぞれのユーザ u に対して訓練を行い, モデルデルタ Δ_{su} を計算し, 以下のように重み付けを行う. ただし, 最初に Δ_{su} を有限体上で(固定小数点へと)近似する必要がある. (論文中のAlgorithm 8, 精度の劣化はほぼない, 例えば 10^{-10} 以下が消える)

$$n_{su} \cdot \Delta_{su} \cdot r_u \cdot Enc\left(\frac{1}{r_u \cdot \sum_{s \in [M]} n_{su}}\right) = Enc\left(\frac{n_{su}}{\sum_{s \in [M]} n_{su}} \cdot \Delta_{su}\right)$$

全てのユーザについて足し合わせる

Paillier暗号の加法準同型性から

$$\Delta_s := \sum_{u \in [N]} Enc\left(\frac{n_{su}}{\sum_{s \in [M]} n_{su}} \cdot \Delta_{su}\right) = Enc\left(\sum_{u \in [N]} \frac{n_{su}}{\sum_{s \in [M]} n_{su}} \cdot \Delta_{su}\right)$$

Protocol (6) 続き

$$\Delta_s := n_{su} \cdot \Delta_{su} \cdot r_u \cdot \text{Enc}\left(\frac{1}{r_u \cdot \sum_{s \in [M]} n_{su}}\right) = \text{Enc}\left(\frac{n_{su}}{\sum_{s \in [M]} n_{su}} \cdot \Delta_{su}\right)$$

実は、この値を計算するためには問題がある。

$\text{Enc}\left(\frac{n_{su}}{\sum_{s \in [M]} n_{su}} \cdot \Delta_{su}\right)$ を計算するためには有限体上 $\mathbb{F}_n$ 上で $\sum_{s \in [M]} n_{su}$ に対する逆元との積が計算されてしまふ(暗号文上の計算自体はPaillier暗号上、つまり $\mathbb{F}_{n^2}$ 上で計算しているが、実際の計算は $\mathbb{F}_n$ 上であることに少し注意)が、これだと、サーバ側で $\sum_{s \in [M]} n_{su}$ なしに、復号した後に正しい値を復元することができない。

なので、ここでトリックとして、1からある $N_{\max}$ までの最小公倍数 LCM を使う

LCMがサーバに対しても公開され、かつ、 $\max_u \sum_{s \in [M]} n_{su} \leq N_{\max}$ (つまり、ユーザー人あたりのレコード数の上界が $N_{\max}$)であれば、

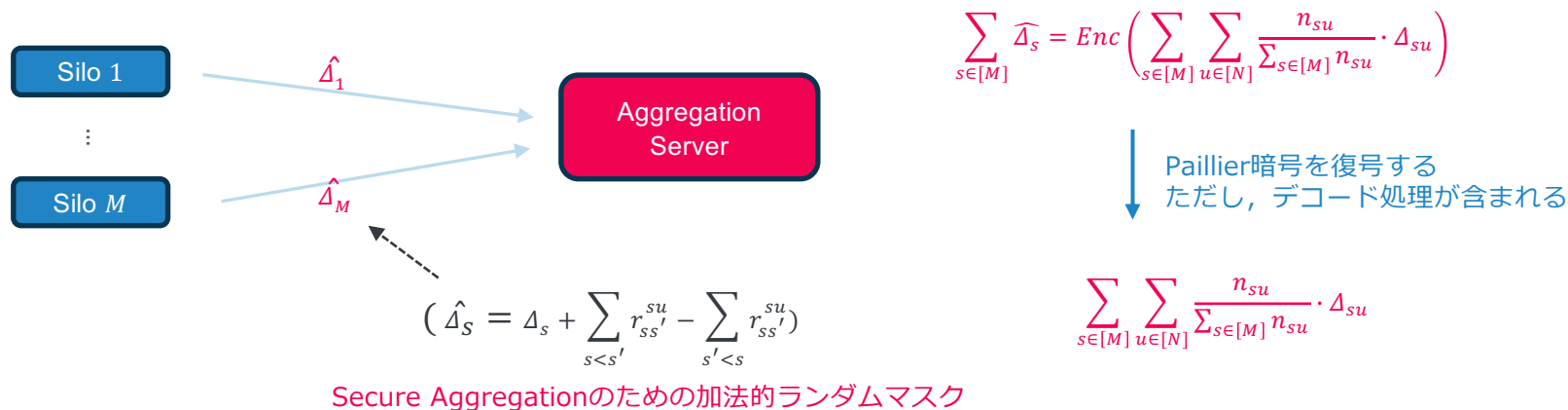
$\text{Enc}\left(\sum_{u \in [N]} \frac{n_{su} \cdot \text{LCM}}{\sum_{s \in [M]} n_{su}} \cdot \Delta_{su}\right)$ を集約して復号した後、 $\sum_{s \in [M]} \left(\sum_{u \in [N]} \frac{n_{su} \cdot \text{LCM}}{\sum_{s \in [M]} n_{su}} \cdot \Delta_{su}\right) / \text{LCM}$ を計算することで、正しい値が計算できる！
($\frac{n_{su} \cdot \text{LCM}}{\sum_{s \in [M]} n_{su}}$ が有限体 $\mathbb{F}_n$ 上で通常の整数空間の計算結果と同じ値になるため)

Paillierのsecurity bitが3072-bitの時、 $N_{\max} = 2000$ でも問題ない

Protocol (7)(8)

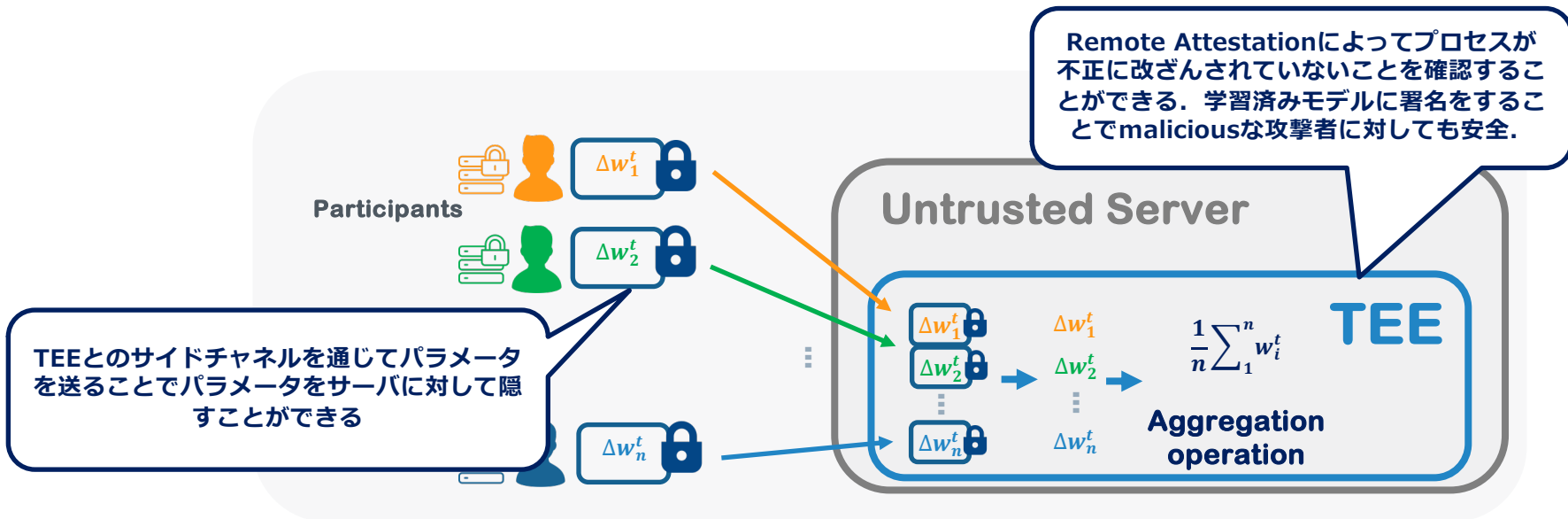
7. (サーバと各サイロ) ステップ4の時と同様に, Secure Aggregation

8. (サーバ) Paillier暗号を復号して, 結果を得る. この際, モデルを有限体上から元の実数上にデコードする操作が必要なので注意(詳細は論文中的アルゴリズム8).



OLIVEに関する付録

サーバサイドTEEを使ったFLのメリット



DP-FLの中で、より厳しい信頼モデルを持つShuffle DPと比べても、より高い有用性を得ることができる

	Trust model	Utility
CDP-FL [4, 28, 50, 84]	Trusted server	Good
LDP-FL [45, 74, 81, 92]	Untrusted server	Limited
Shuffle DP-FL [29, 44]	Untrusted server + Shuffler	Shuffle DP-FL $\leq$ CDP-FL
OLIVE (Ours)	Untrusted Server with TEE	OLIVE = CDP-FL

Prime+Probeの概要

- ▶ 興味対象のキャッシュセット領域を自身のデータで埋め尽くす (Prime)
- ▶ 攻撃対象が活動する時間として一定時間待つ
- ▶ 興味対象のキャッシュセット領域へアクセス (Probe)
- ▶ もし 3 でのアクセス時間が長ければ、キャッシュが無くなったと解釈し、「2 の一定時間で攻撃対象が興味対象のキャッシュセットを利用した」と考えられる

<https://milestone-of-se.nesuke.com/sv-advanced/sv-security/cache-side-channel/> より

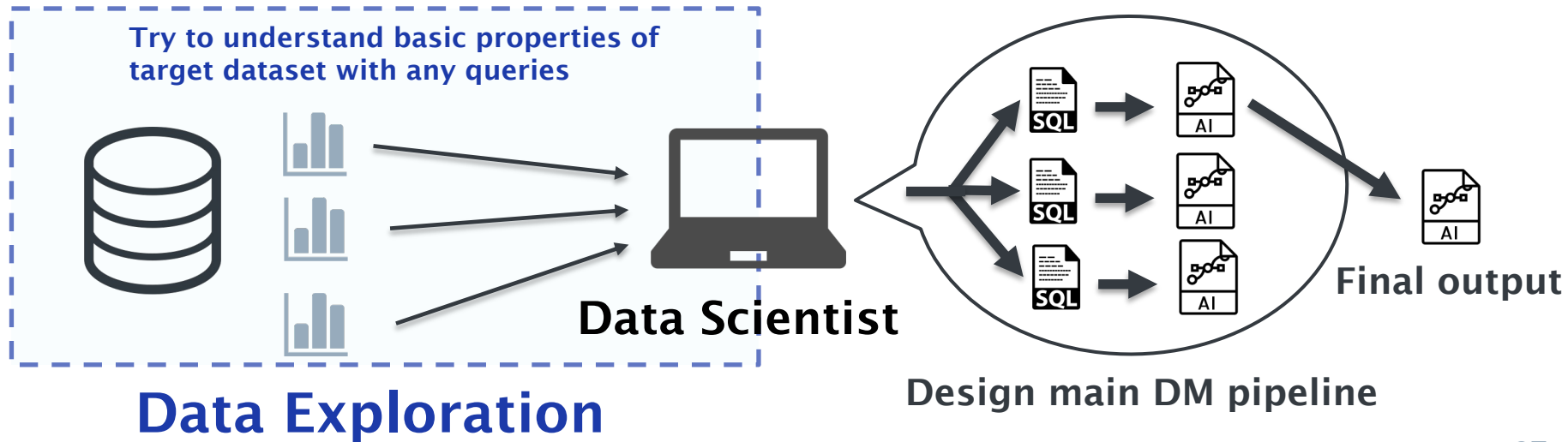
[研究成果A] HDPView

HDPView: Differentially Private Materialized View for Exploring High Dimensional Relational Data



Data Exploration (DE)

- What is data exploration? -- Early stage of data mining workflow
 - Data scientist designs a data mining workflow based on the properties of target dataset
 - If the data is sensitive is this data exploration possible?**



Summary: DP view for data exploration

Q. How can we construct a privacy-preserving view to explore the high-dimensional (e.g., 20D) sensitive data?

Non-private
Relational data

Name	Age	Salary	...
Alice	22	2M	...
...	...	...	...
Bob	23	1M	...
Charlie	34	4M	...
David	38	3M	...
Emma	42	12M	...
Fumi	47	10M	...
...	...	...	...

Our algorithm

Differentially Private view

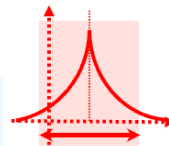
Block ID	Age	Salary	...	Noisy Count
1	[20,24]	[1M,2M]	...	$202 + z_1$
2	[34,38]	[3M,5M]	...	$155 + z_2$
3	[40,49]	[10M,12M]	...	$567 + z_3$
...	...	...	...	...

N-dimensional **noisy Histogram** covering the entire domain

Data Exploration via any counting queries

query
←→
...
←→

Data Scientist



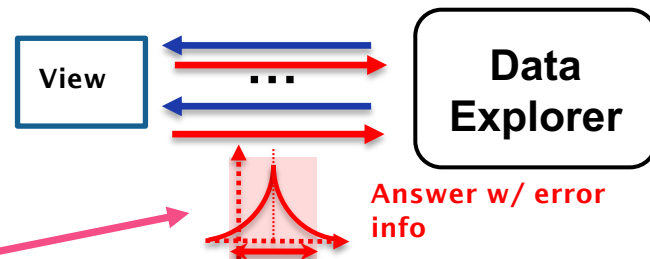
Noisy count w/ error information

Requirements for DE under DP (1)(2)

- Requirement (1): **Workload independence**

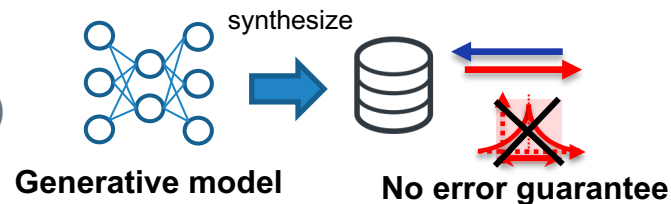
- Issuable query set should be unlimited, and not pre-defined
- (x) Workload optimization methods (e.g., *HDMM*[1])

Unlimited queries



- Requirement (2): **Analytical reliability**

- Estimatable scale of the error for any counting queries
- (x) Generative model approach (e.g., *Privbayes*[2], *DP-GAN*[3])



[1] R. McKenna, et.al.,. Optimizing error of high-dimensional statistical queries under differential privacy. Proc. VLDB Endow., 11(10):1206–1219, June 2018.
[2] J. Zhang, et.al.,. Privbayes: Private data release via bayesian networks. ACM Transactions on Database Systems (TODS), 42(4):25, 2017.
[3] J. Fan, et.al.,. Relational data synthesis using generative adversarial networks: A design space exploration. Proc. VLDB Endow., 13(12):1962–1975, July 2020.

Requirements for DE under DP (3)(4)

- Requirement (3): **Noise resistance on high-dimensional data**
 - Explore high-dimensional relational data with less noise
 - (×) Existing partitioning based methods (e.g., *Privtree*[4], *DAWA*'s partition[5])
- Requirement (4): **Space efficient view**
 - View to be explored should be space-efficient

e.g., 20 columns

**Non-private
Relational data**

Name	Age	Salary	...
Alice	22	2M	...
...	...	...	...
Bob	23	1M	...
Charlie	34	4M	...
David	38	3M	...
Emma	42	12M	...
Fumi	47	10M	...
...	...	...	...

Differentially Private view

Block ID	Age	Salary	...	Noisy Count
1	[20,24]	[1M,2M]	...	$202 + z_1$
2	[34,38]	[3M,5M]	...	$155 + z_2$
3	[40,49]	[10M,12M]	...	$567 + z_3$
...	...	...	...	...

[4] Zhang, Jun, et.al., Privtree: A differentially private algorithm for hierarchical decompositions. Proceedings of the 2016 SIGMOD. 2016.

[5] C. Li, et.al., A data-and workload-aware algorithm for range queries under differential privacy. Proceedings of the VLDB Endowment, 7(5):341–352, 2014.

Partitioning approach

- Take relational input data as n-dimensional histogram,

$$z_i \sim \text{Laplace}\left(\frac{\Delta f}{\epsilon}\right)$$

Original + noise

Salary	~10M	20M	30M	40M
Age ~20	$1+z_1$	$0+z_2$	$5+z_3$	$4+z_4$
20~30	$2+z_5$	$1+z_6$	$8+z_7$	$7+z_8$
30~40	$13+z_9$	$62+z_{10}$	$64+z_{11}$	$0+z_{12}$
40~50	$0+z_{13}$	$0+z_{14}$	$1+z_{15}$	$1+z_{16}$

Partitioning

Partitioned + noise

$4+z_1$		$24+z_2$	
$13+z_3$	$126+z_4$		$0+z_5$
$2+z_6$			

Query: SELECT COUNT ...

$$174+Z, Z \text{ stddev} = 16 \frac{\Delta f}{\epsilon}$$

$$174+Z', Z' \text{ stddev} = 6 \frac{\Delta f}{\epsilon}$$

Perturbation Error (PE) can be reduced

Partitioning as a View

- Considering partitioned blocks as a **materialized view**, which can satisfy req. (1)

Partitioned blocks

Salary	~10M	20M	30M	40M	
Age	~20	$4+z_1$		$24+z_2$	
	20~30				
	30~40	$13+z_3$	$126+z_4$		$0+z_5$
	40~50	$2+z_6$			

=

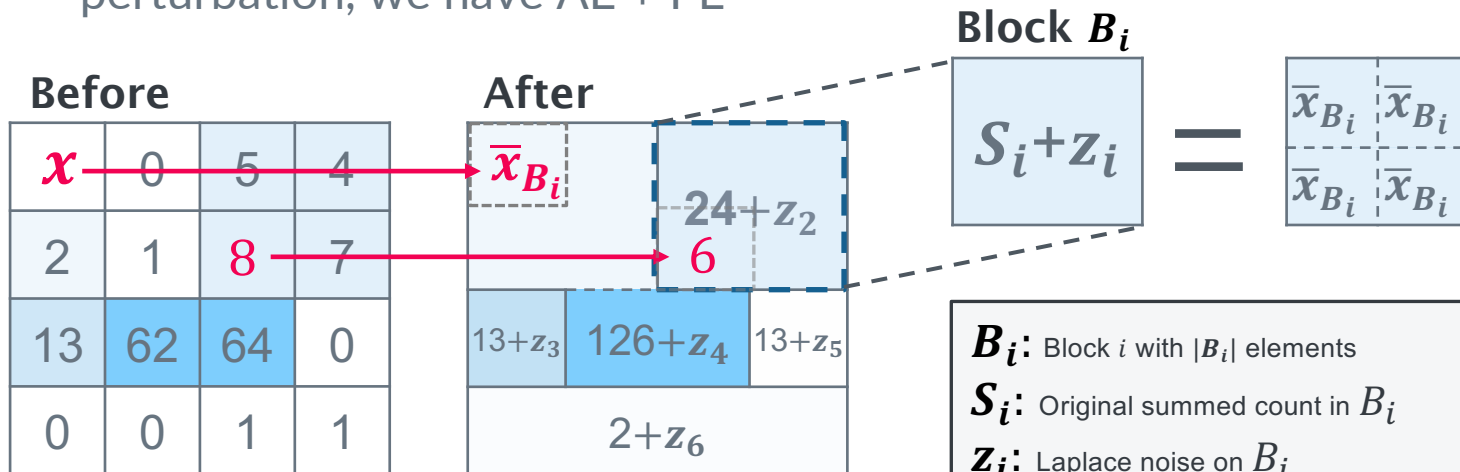
$1 + \frac{z_1}{4}$	$1 + \frac{z_1}{4}$	$6 + \frac{z_2}{4}$	$6 + \frac{z_2}{4}$
$1 + \frac{z_1}{4}$	$1 + \frac{z_1}{4}$	$6 + \frac{z_2}{4}$	$6 + \frac{z_2}{4}$
$13 + z_3$	$63 + \frac{z_4}{2}$	$63 + \frac{z_4}{2}$	$0 + z_5$
$0.5 + \frac{z_6}{4}$	$0.5 + \frac{z_6}{4}$	$0.5 + \frac{z_6}{4}$	$0.5 + \frac{z_6}{4}$

req.(1): Can answer any range counting queries

WHERE $20 \leq \text{Age} \leq 30$ AND Salary=20 $\Rightarrow 64.5 + \frac{z_1}{4} + \frac{z_4}{2} + \frac{z_6}{4}$

Aggregation and Perturbation Error

- Partitioning causes aggregation error (AE), so after partition and perturbation, we have AE + PE



$$|\mathbf{x} - \bar{\mathbf{x}}_{B_i}| = \underbrace{\left| \mathbf{x} - \frac{S_i}{|B_i|} \right|}_{\text{Aggregation Error (AE)}} - \underbrace{\frac{z_i}{|B_i|}}_{\text{Perturbation Error (PE)}}$$

Error of $\mathbf{x}$

B_i : Block i with $|B_i|$ elements
 S_i : Original summed count in B_i
 z_i : Laplace noise on B_i
 $\mathbf{x}$: Original counts
 $\bar{\mathbf{x}}_{B_i} = \frac{1}{|B_i|} (S_i + z_i)$

Error Optimization

Goal: Minimization

- We can formulate multi-dim partitioning as optimization problem

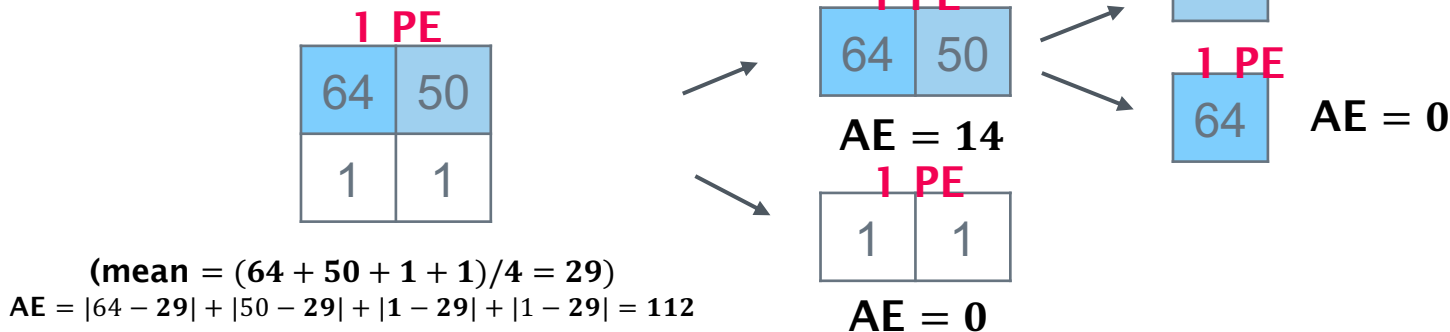
$$\mathbb{E} \left[\sum_{i \in [m]} \sum_{x \in B_i} |\mathbf{x} - \bar{\mathbf{x}}_{B_i}| \right] \leq \mathbb{E} \left[\sum_{i \in [m]} AE(B_i) \right] + \mathbb{E} \left[\sum_{i \in [m]} PE(B_i) \right] \leq \sum_{i \in [m]} \left(AE(B_i) + m \cdot \frac{1}{\epsilon} \right)$$

- An instance of the set partitioning problem: **NP-Hard**
- Search algorithm for the optimal partitioning **must satisfy DP**

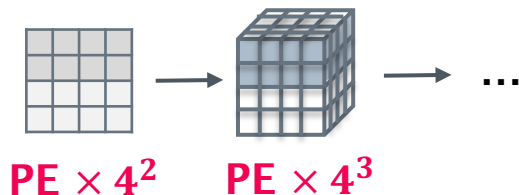
We need to consider heuristic solutions to obtain better partitioning, and it must be simple enough to allow DP analysis.

Heuristics

- One main heuristic is *splitting can reduce AE*



- Decrease the number of blocks is more important in High-dimensional setting
 - The domain of n -dimensional histogram increases exponentially



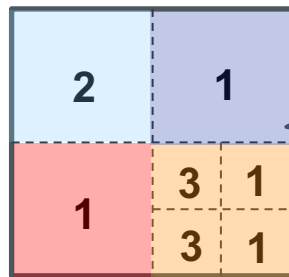
*Differences from
previous studies*

Existing Approach: Privtree [4]

- Recursive splitting by Quadtree for efficient and fine partitioning

Problem

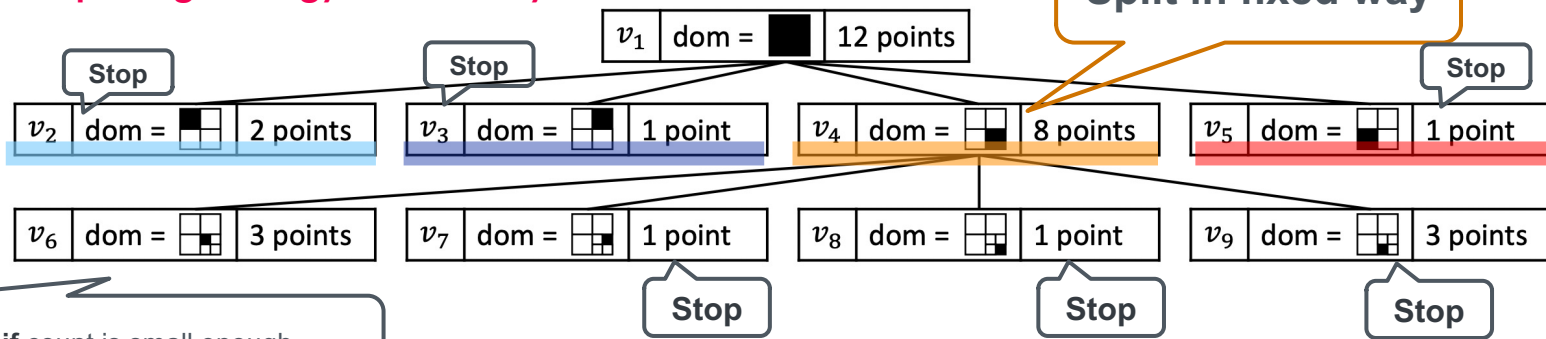
Too fine-grained in High dimension
→ Too large PE (3) and space-inefficient (4)



Quadtree-based recursive splitting until count is small enough

More careful splitting strategy is necessary

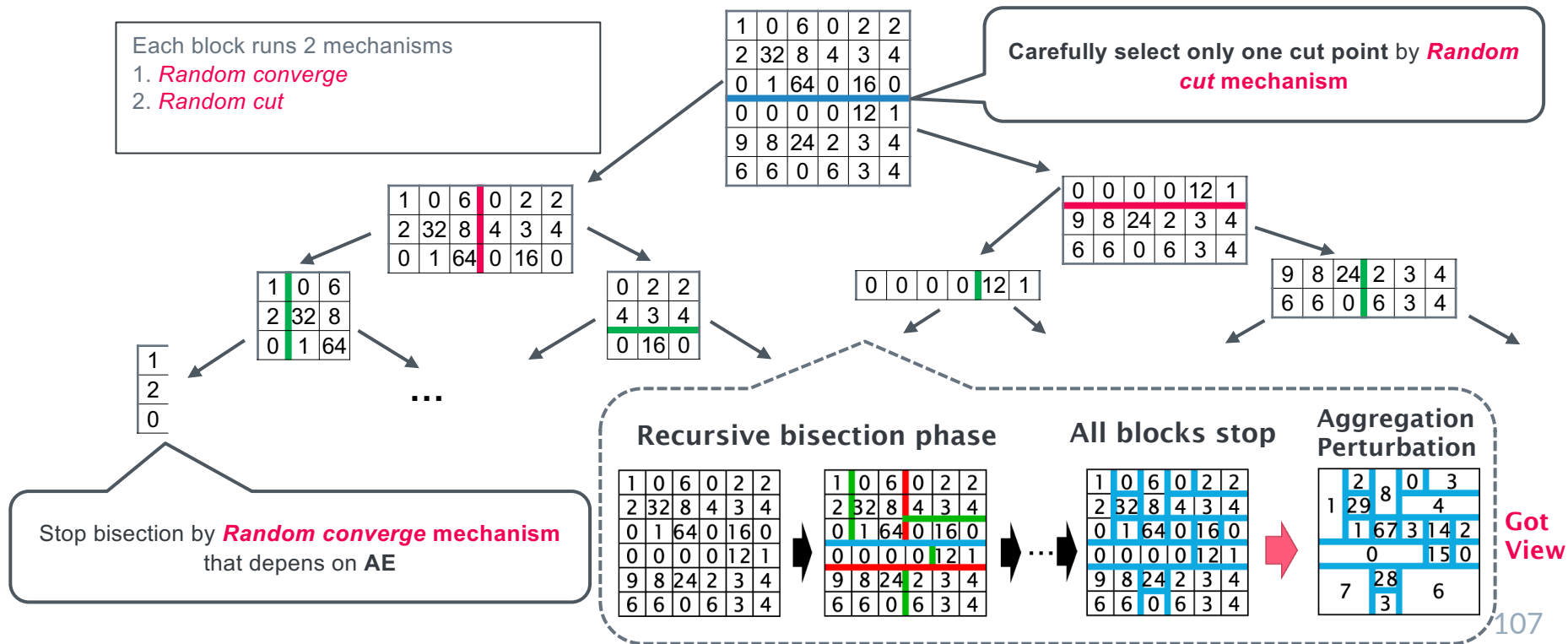
Split in fixed way



Stop if count is small enough

HDPView (Proposed method)

- Recursive **bisection**-based differentially private search algorithm



Random cut, Random converge

- Random cut

$$\Pr[p^* = p] \sim \exp\left(\frac{\epsilon Q(B, p)}{2\Delta_Q}\right)$$

$$Q(\mathcal{B}, p) = -(\text{AE}(\mathcal{B}_L^{(p)}) + \text{AE}(\mathcal{B}_R^{(p)}))$$

- Random converge

if $\text{AE}(\mathcal{B}) + \text{Lap}(\lambda) \leq 1/\epsilon_p$
then, stop the recursive bisection

Aggregation
Perturbation

	2	8	0	3
1	29			4
	1	67	3	14
		0		15
7	28			6
	3			

We analyze a **DP guarantee on the final view** and an approximated error for the any queries on the view. (see our paper in detail)

req.2

Save unnecessary split

Only one cut point p is probabilistically selected from all possible points using exponential mechanism

Smaller total AE after bisection is more likely to be sampled

Effective split for small AE

Stop if Laplace noised AE of block B is small enough

$1/\epsilon_p$ is stddev of noise. If $\text{AE} < 1/\epsilon_p$, $\text{AE} + \text{PE}$ will worsen by any bisection.

Experimental Settings

- **Task:** 8 workloads of range counting queries
 - 1-way All range, {2,3}-way Marginal, {2,3}D-Prefix, {2,3,4}D-Random range
 - **Metric:** *RMSE (Root mean squared error)*

- **Competitors**

- *Identity, HDMM, Privtree, Privbayes*

- **Dataset:** 8 real-world datasets

Dataset	#Record	#Column (categorical)	#Domain	Variance
Adult [1]	48842	15 9)	9×10^{19}	0.0360
Small-adult	48842	4 2)	3×10^5	0.0237
Numerical-adult	48842	7 1)	2×10^{11}	0.0200
Traffic [5]	48204	8 2)	1×10^{14}	0.0484
Bitcoin [6]	500000	9 1)	4×10^{12}	0.0379
Electricity [2]	45312	8 1)	1×10^{14}	0.0407
Phoneme [4]	5404	6 1)	2×10^6	0.0304
Jm1 [3]	10885	22 1)	2×10^{21}	0.0027

Dimension

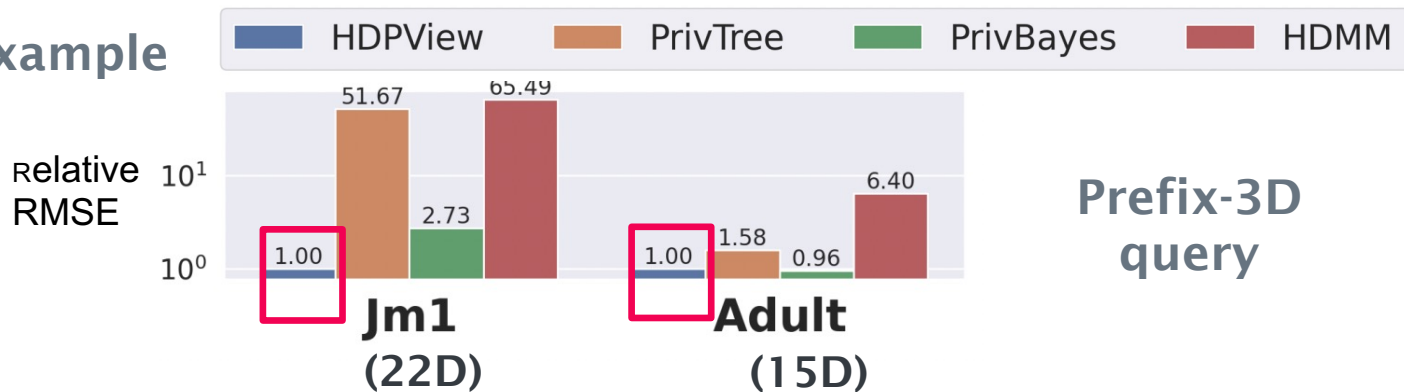
Noise resistance on HD data

- Average relative RMSE (ARR) over all 8 workloads and 8 datasets

	Identity	Privtree	HDMM	Privbayes	HDPView (ours)
ARR	1.94×10^7	7.05	35.34	3.79	1.00

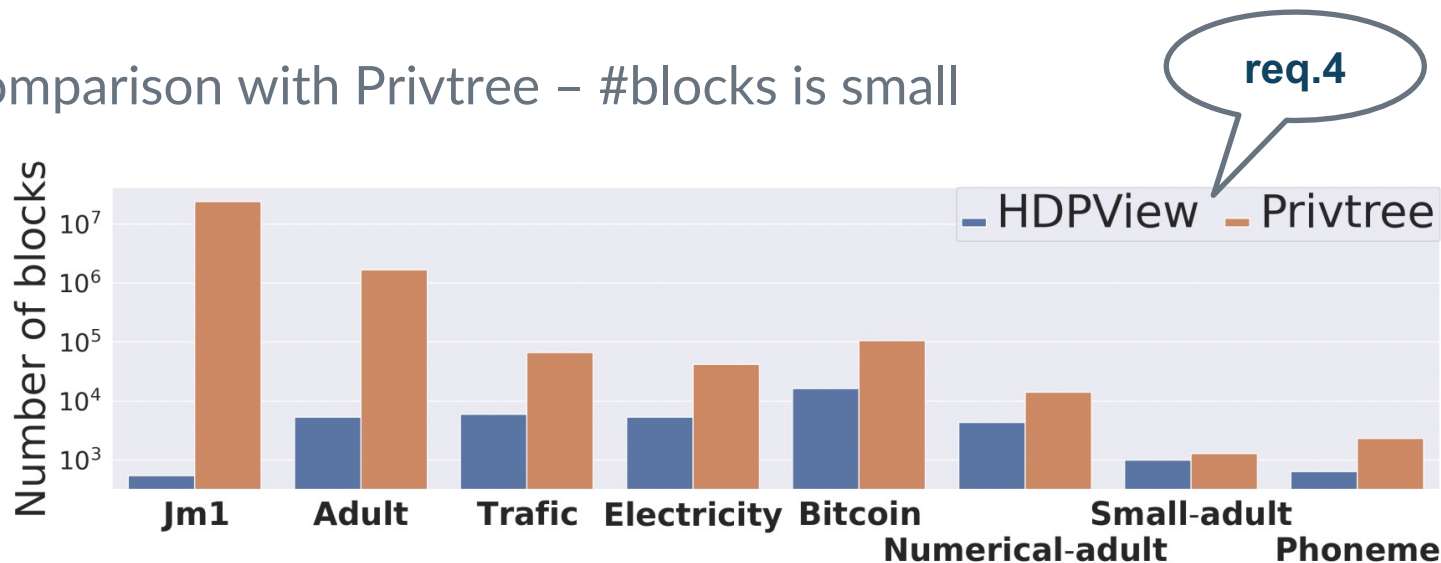
req.3

Example



Space-efficient

- Comparison with Privtree – #blocks is small



- With naïve Laplace over all domain

Dataset	Identity-based	HDPVIEW
Adult	30.99 EB	3.61 MB
Bitcoin	1.27 TB	6.77 MB
Electricity	1.11 TB	2.19 MB
Phoneme	781.34 KB	273.59 KB

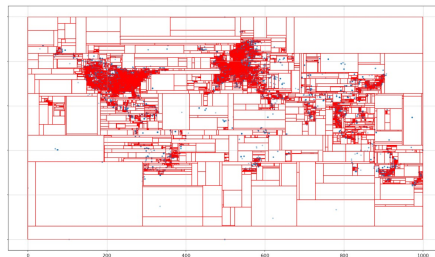
Conclusion

- Q: How can we construct a privacy-preserving view to explore the high-dimensional sensitive dataset?
 - We propose HDPView, which create a high-dimensional differentially private view by properly balancing AE and PE for high-dimensional data
 - The experiments show HDPView's noise resistance and space-efficiency compared to existing works

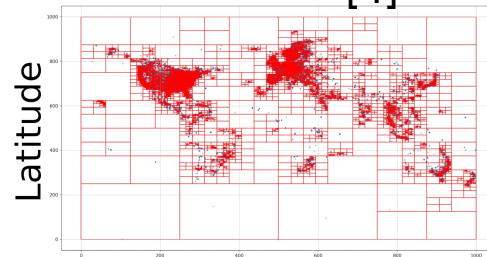
Characteristics

- Flexible block partitioning

HDPView



Privtree[4]



- Better convergence

- Carefully cutting **one point** at a time, the number of generated blocks can be reduced, which reduces PE and creates space-efficient view
- Histograms of the real dataset are very sparse and require few blocks

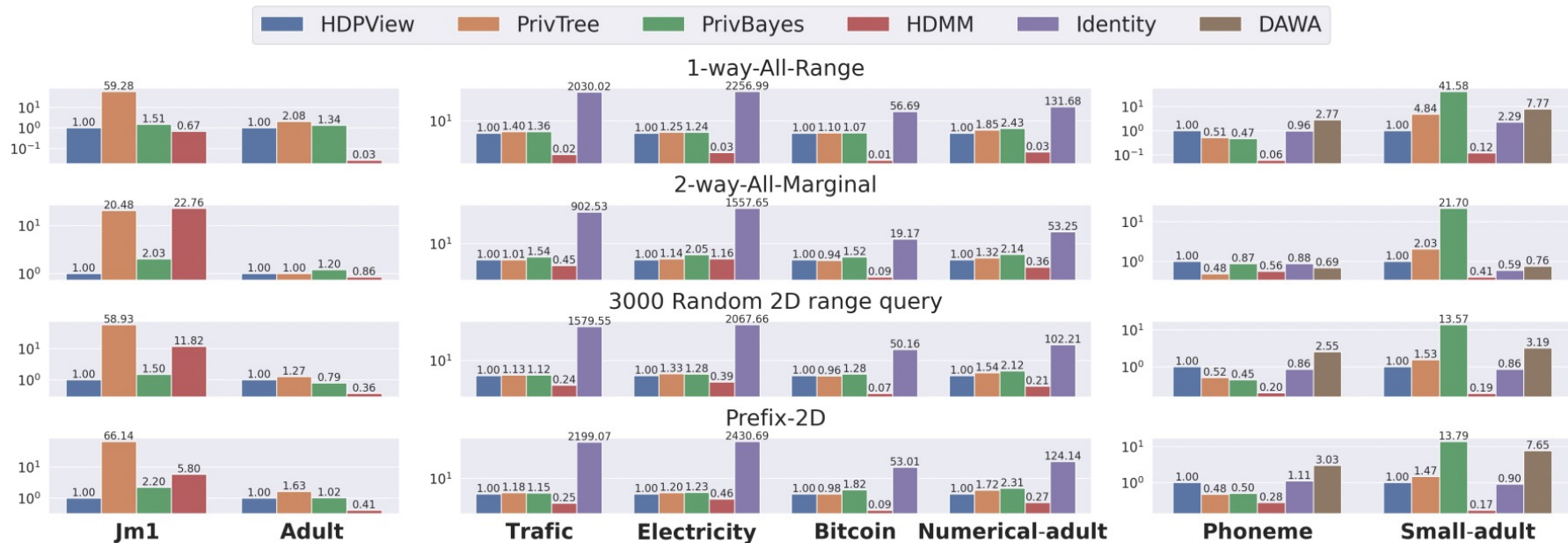
- Dimensional scalability

- Similar to *Mondrian* [6] for K-anonymity
- The algorithm execution is not affected much by the increase in dimensionality

Visualization on 2D Data (Left is ours)

[6] LeFevre, Kristen, David J. DeWitt, and Raghu Ramakrishnan. "Mondrian multidimensional k-anonymity." 22nd International conference on data engineering (ICDE'06). IEEE, 2006.

Range counting queries (1,2-D)



Range counting queries (3,4-D)

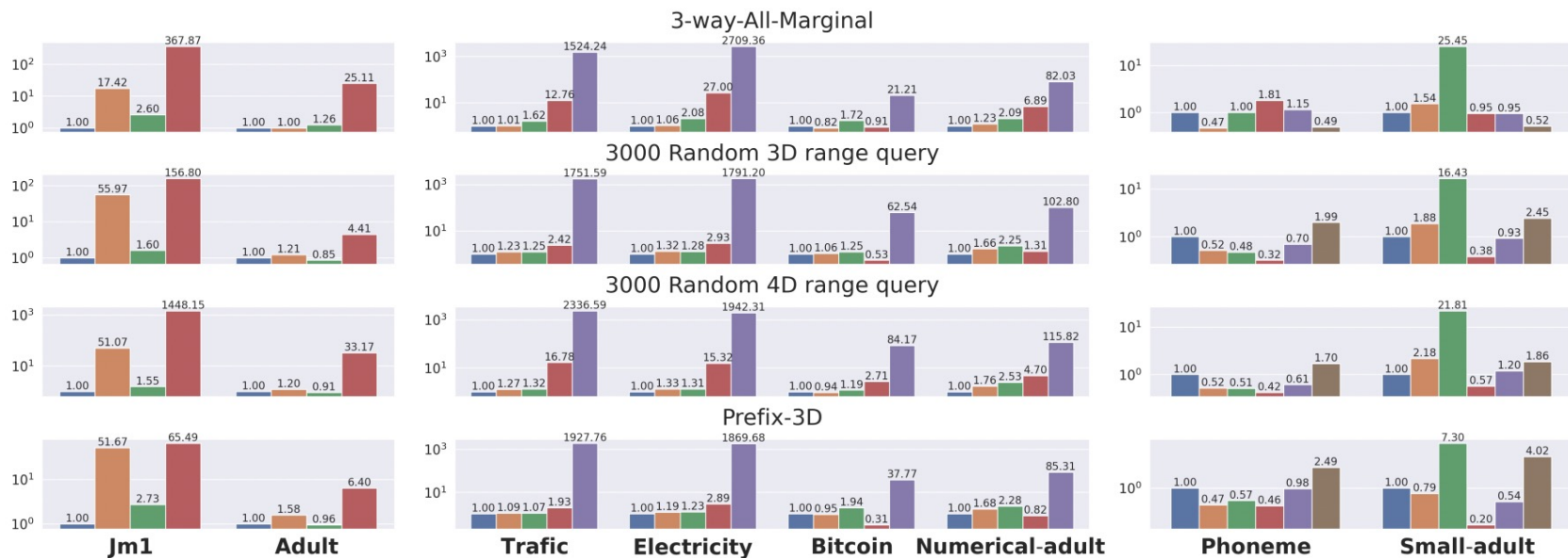


Figure 3: Relative RMSEs against HDPVIEW on all the datasets and workloads: HDPVIEW shows small errors for a wide variety of high-dimensional range counting queries.

Comparison with round robin Privtree

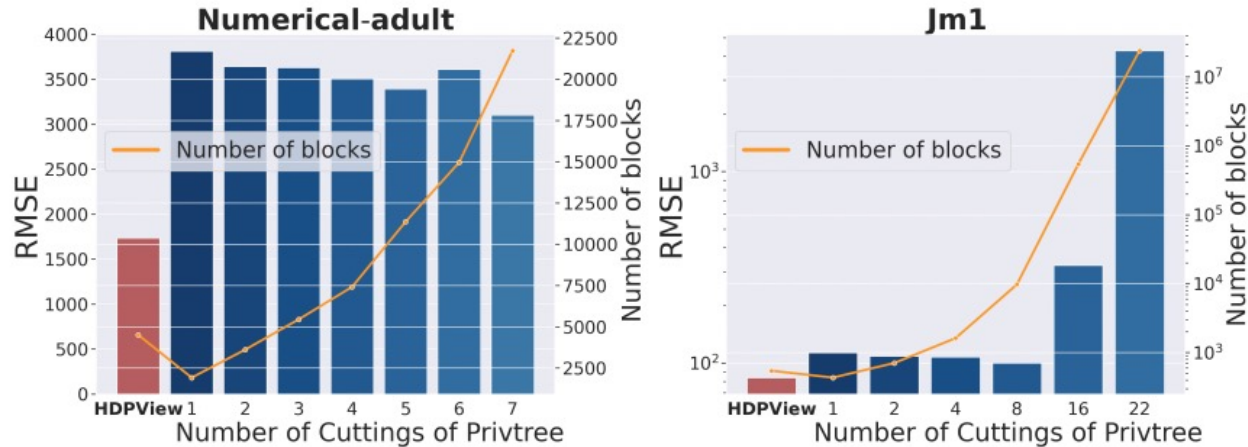
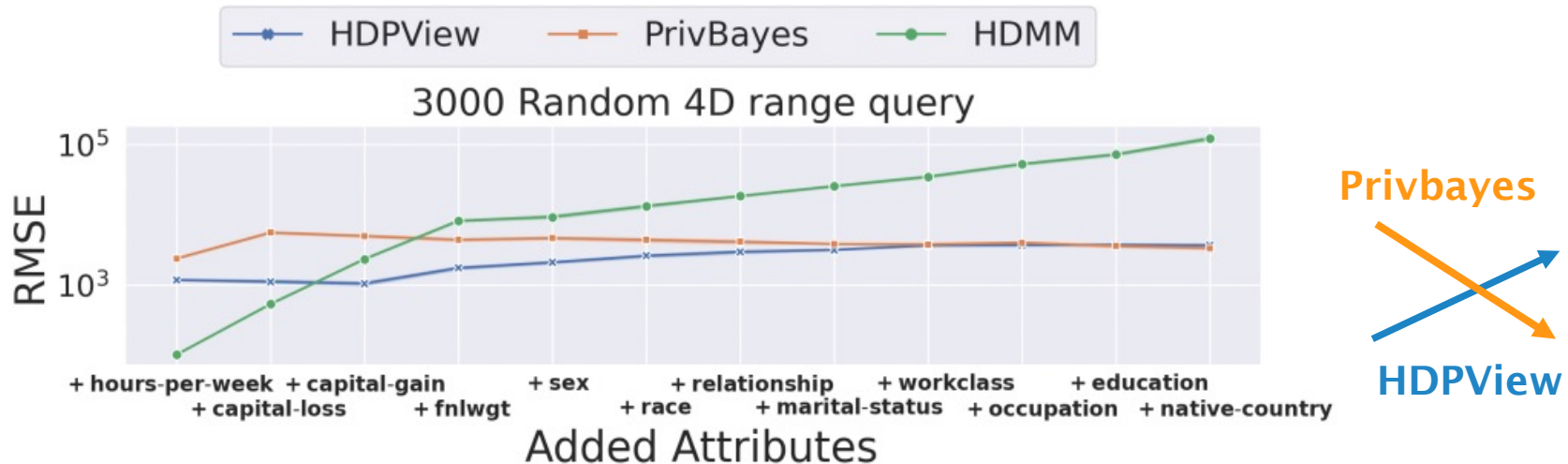


Figure 6: HDPVIEW is more effective than Privtree even with controlled number of cuttings on Numerical-adult (left) and Jm1 (log-scale) (right).

Affects of dimensionality

- Changes in the performance when adding attributes to Adult one by one in HDPView, PrivBayes, and HDMM
 - HDPView's performance degrades slowly but surely with increasing dimensionality, while PrivBayes' performance rather improves



[研究成果B] PCT-TEE

PCT-TEE: Trajectory-based Private Contact Tracing System with Trusted Execution Environment

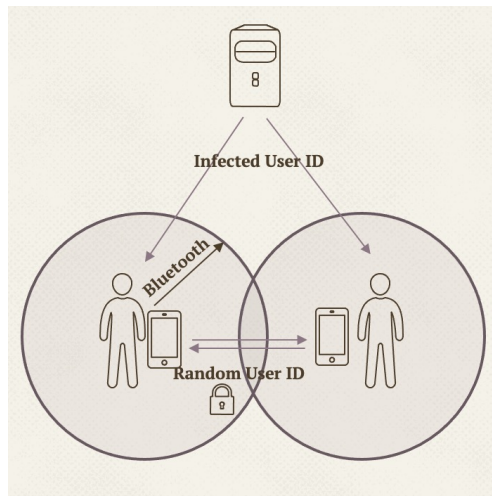
ストーリー

▷ 既存の Contact Tracing 手法(BLEベース)の欠点

- 接触判定の時間・空間的な制約
 - 間接的な接触が発見できないなど
- 接触判定ロジックを変更できない

▷ 軌跡データを収集し秘密計算する

- 暗号ベースの Private Set Intersection 手法: スケールしないのでTEEを利用
- Intel SGX: 強力な**TEE**だけど現在のところリソース制約が厳しい, その中でどのようにスケーラブルなContact Tracingを実現するかを考えた



Spatio-temporal PSI

- ▷ (効率化のために) 接触判定をPSIに近似する

$$\begin{cases} 1 & (\exists A^{(u)} \cap A^{(v)} \neq \emptyset \text{ s.t.} \\ & A^{(u)} = \{f_{\Theta}(x_i^{(u)}) \mid x_i^{(u)} \in X_u\} \text{ and } A^{(v)} = \{f_{\Theta}(x_j^{(v)}) \mid x_j^{(v)} \in X_v\}) \\ 0 & (\text{others}) \end{cases}$$

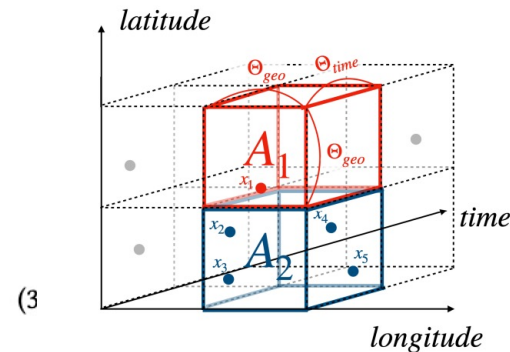
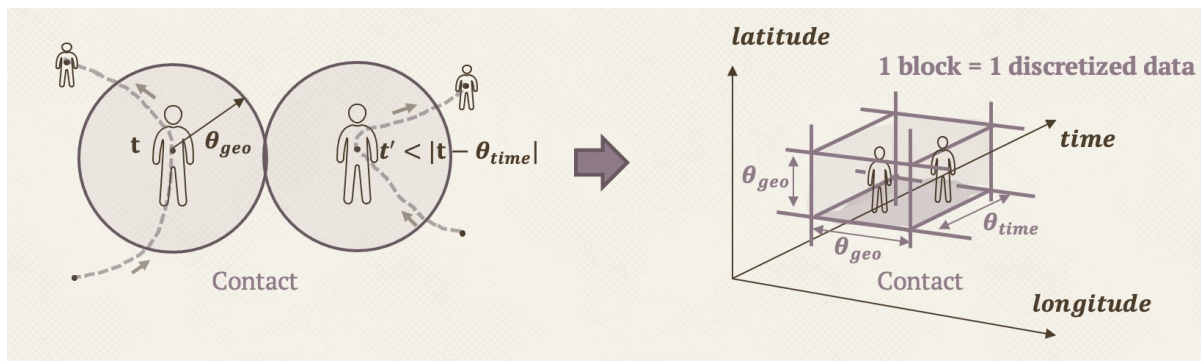


Fig. 2. Spatiotemporal Private Set Intersection.



データの流れ

- 保健所が陽性者の軌跡データをサーバに登録
 - サーバ上のTEEの中でのみ復号
- クライアント（ユーザ）は接触していないかを確認するためにサーバにクエリを送信
 - クエリのパラメータは自分の軌跡データ
 - サーバのTEEの内部でのみ復号
- サーバは軌跡データのマッチングを計算して結果を返却
 - 結果は暗号化される

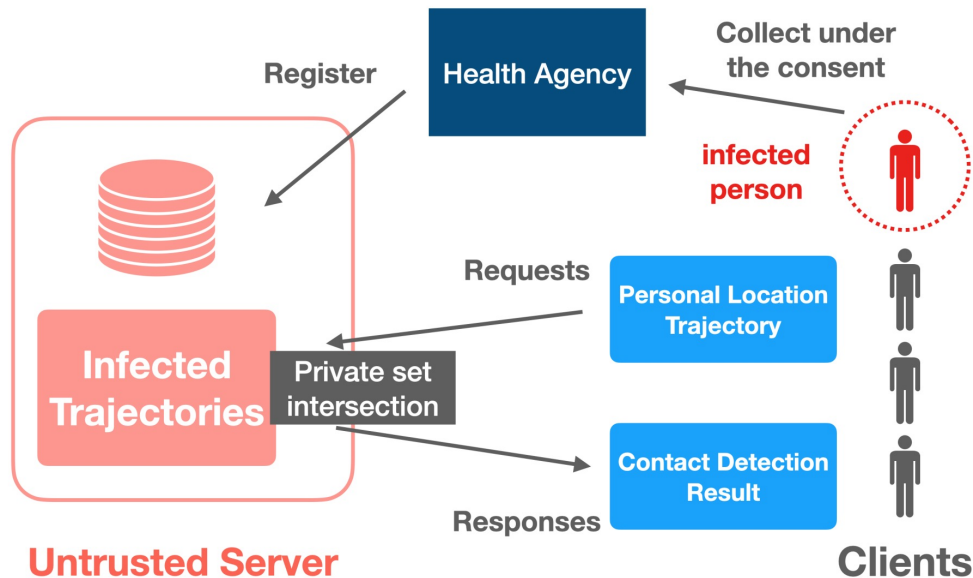


Fig. 1. Trajectory-based PCT overview.

提案システム概要

- 陽性者の軌跡データはバッチ更新
- クライアントはSGXに対して Remote Attestationによって鍵交換を行う
- 陽性者の軌跡データは大きいのでチャンク化して実行時にTEEのメモリに繰り返しロードしてクライアントの軌跡データとマッチングを計算
- 結果を暗号化して返す

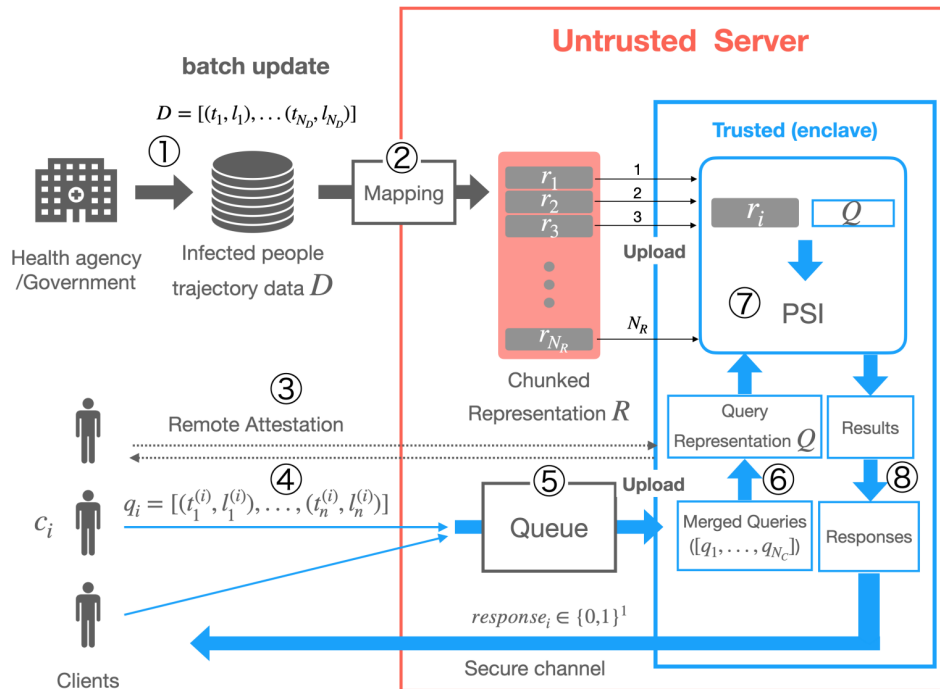


Fig. 3. Architecture overview: circled numbers correspond to the steps shown in *System Overview*.

ポイント

- ▶ サーバ側の軌跡データの圧縮
 - SGXのメモリ制約が厳しいため
- ▶ 高速なマッチング
 - 軌跡データの量が多いため
- ▶ 軌跡データを階層的なエンコードとデータ構造によって圧縮と高速なマッチングを同時に解決

軌跡データからマッチングまで

クライアント側: 軌跡データをエンコード (TrajectoryHash)

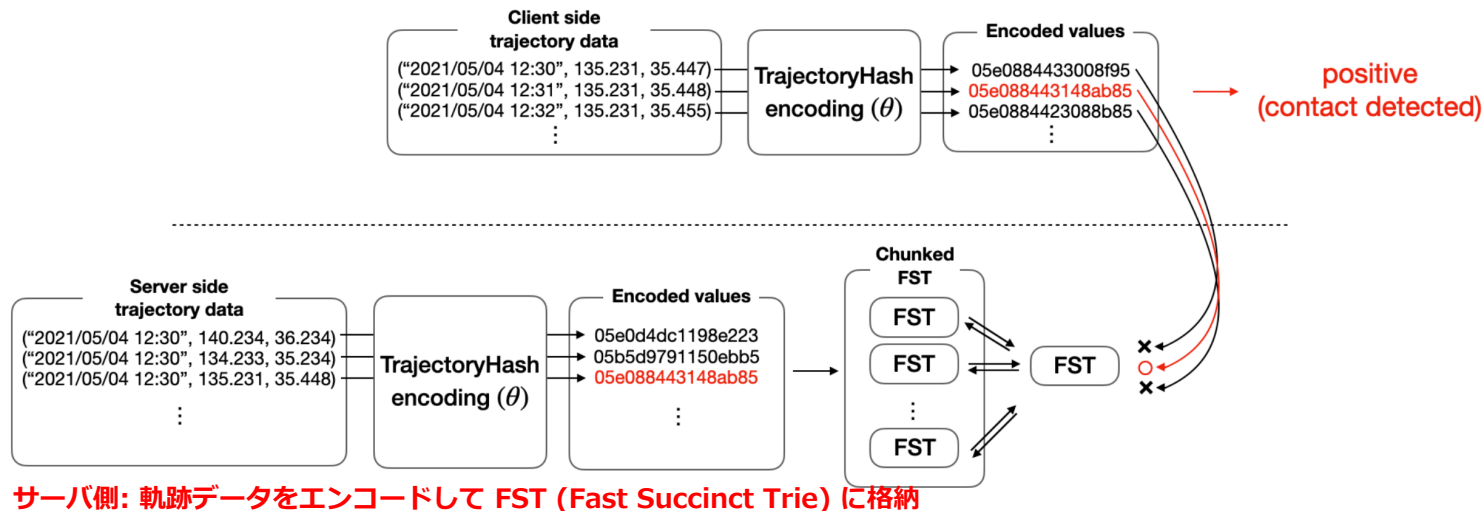


Fig. 4. An overview of running example from raw trajectory to the final risk assessment.

軌跡データからマッチングまで

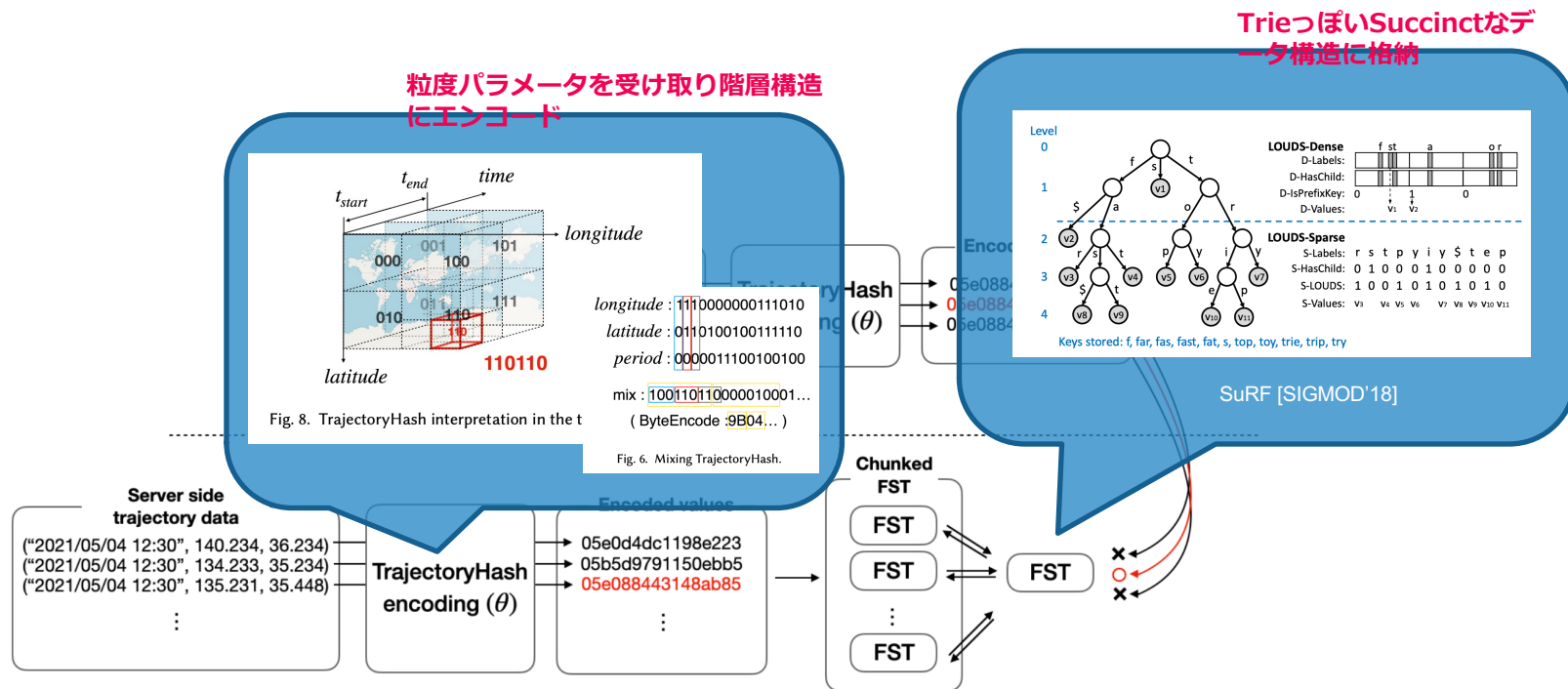
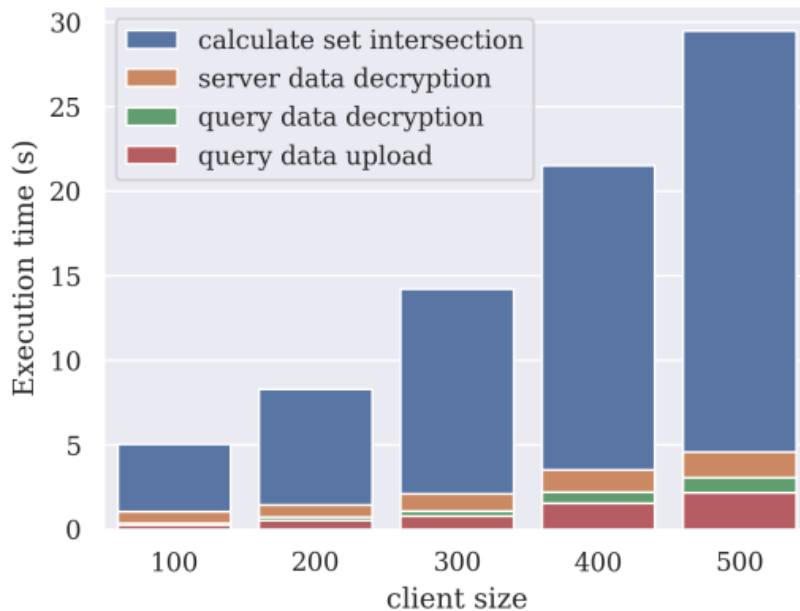


Fig. 4. An overview of running example from raw trajectory to the final risk assessment.

実験結果

- ▷ 設定
 - 1分ごとに2週間分のデータ
 - 20160個 = 60 分*24 時間*14 日
- ▷ 陽性者
 - 5000人
- ▷ クライアント
 - 100 ~ 500人
 - キューイングしてまとめて処理している



このくらいの規模感で20人 / 秒くらいのスループット